



TRƯỜNG ĐẠI HỌC SÀI GÒN

Khoa Công nghệ thông tin

Báo cáo lần 2

DATA BRIDGE – HỆ THỐNG TRUY VẤN DỮ LIỆU THÔNG MINH BẰNG NGÔN NGỮ TỰ NHIÊN

GV hướng dẫn: TS. Đỗ Như Tài
Lớp: DCT122C3 – Mã học phần: 841482
Sinh viên: Lê Hồng Phát – 3122411145
Trương Phú Kiệt – 3122411109
Trà Đức Toàn – 3122411217
Lê Ngọc Hiệp – 3122411053

Thành phố Hồ Chí Minh, tháng 5 năm 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan báo cáo về đề tài này là công trình nghiên cứu và phát triển của cả nhóm dưới sự hướng dẫn và góp ý của **TS. Đỗ Như Tài**.

Tất cả đều được thực hiện dựa trên quá trình học tập, nghiên cứu và thực nghiệm thực tế. Các số liệu, kết quả nêu trong báo cáo là trung thực và chưa từng được công bố trong bất kỳ công trình của tác giả nào khác. Các tài liệu tham khảo đều được trích dẫn nguồn gốc rõ ràng theo quy định.

Chúng em xin hoàn toàn chịu trách nhiệm trước Nhà trường và pháp luật về tính xác thực của lời cam đoan này.

Xin chân thành cảm ơn.

Sinh viên thực hiện

Trương Phú Kiệt

LỜI CẢM ƠN

Để hoàn thành dự án này, bên cạnh sự nỗ lực của cả nhóm, em và các thành viên trong nhóm đã nhận được rất nhiều sự quan tâm, giúp đỡ giảng viên và bạn bè.

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành sâu sắc nhất đến Trường đại học **Sài Gòn – Khoa Công nghệ thông tin** là nơi mà đã truyền cảm hứng và những nền tảng kiến thức vững vàng, là hành trang quan trọng nhất để chúng em tiếp tục trên con đường phát triển.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn chân thành nhất đến thầy **Đỗ Như Tài**, người đã tận tâm hướng dẫn, và truyền đạt những kiến thức bổ ích đến với sinh viên chúng em. Cuối cùng, chúng em xin cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện tốt nhất về tinh thần để tôi tập trung hoàn thành báo cáo.

Trong quá trình nghiên cứu và phát triển, chúng em đã cố gắng để hoàn thành những đầy đủ yêu cầu của đề tài thì chắc chắn vẫn có những thiếu sót không thể tránh khỏi. Chúng em mong nhận được những đóng góp, nhận xét quý giá từ Thầy và các bạn để công trình nghiên cứu này phát triển hơn.

Xin chân thành cảm ơn.

Sinh viên thực hiện

Trương Phú Kiệt

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
CHỮ VIẾT TẮT	vi
DANH MỤC HÌNH ẢNH	vii
BẢNG PHÂN CÔNG	viii
TÓM TẮT ĐỀ TÀI BÁO CÁO	ix
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1. Tính cấp thiết của đề tài.....	1
1.2. Tổng quan tình hình nghiên cứu.....	1
1.3. Mục tiêu nghiên cứu.....	2
1.4. Nhiệm vụ nghiên cứu.....	3
1.5. Phạm vi và đối tượng nghiên cứu	5
1.6. Phương pháp nghiên cứu	5
1.7. Ý nghĩa khoa học và thực tiễn	6
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	8
2.1. Mô hình ngôn ngữ lớn (LLM) và các tương tác	8
2.1.1. Sự tích hợp đa dạng các Mô hình ngôn ngữ lớn.....	8
2.1.2. Kỹ nghệ câu lệnh (Prompt Engineering) và Chuẩn hóa JSON.....	8
2.1.3. Kỹ thuật Grounding và Cơ chế giảm thiểu Ảo giác AI (Hallucination)	9
2.1.4. Lớp ngữ nghĩa (Semantic Layer) và Tìm kiếm tăng cường ngữ cảnh (RAG).....	9
2.2. Bài toán NL2SQL và Multi-Agent	10
2.2.1. Tổng quan Kiến trúc Đa tác vụ (Multi-Agent Pipeline)	11
2.2.2. Tác nhân Phân tích Ý định (Architect Agent)	11
2.2.3. Tác nhân Sinh mã lệnh (SQL Generator Agent).....	12
2.2.4. Tác nhân Kiểm định và Sửa lỗi (Validator Agent).....	12
2.2.5. Các quy tắc an toàn cốt lõi trong luồng Đa tác vụ	12
2.3. Kiến trúc Microservices và Backend	13
2.3.1. Các phân hệ vi dịch vụ lõi.....	13
2.3.2. Công nghệ phát triển API (FastAPI & Pydantic)	13
2.3.3. Quản lý kết nối cơ sở dữ liệu với Psycopg2.....	13
2.4. Hệ quản trị cơ sở dữ liệu và Cơ chế bảo mật Supabase.....	14
2.4.1. Nền tảng Supabase PostgreSQL và Cơ chế Connection Pooling	14

2.4.2. Lớp bảo mật đa tầng và Chống tiêm nhiễm mã độc (SQL Injection) ..	14
2.4.3. Cơ chế cường chế Chỉ đọc (Read-Only) và Giới hạn tài nguyên	15
2.4.4. Bảo mật cấp độ hàng (RLS) và Quản trị quyền truy cập	15
2.5. Công nghệ phát triển Giao diện người dùng (Frontend).....	16
2.5.1. Nền tảng Công nghệ cốt lõi (Core Tech Stack).....	16
2.5.2. Phong cách Thiết kế Modern SaaS và Hiệu ứng Glassmorphism	16
2.5.3. Hệ thống Kiểu chữ (Typography) và Bố cục không gian (Pixel-Perfect Spacing)	17
2.5.4. Xử lý Hoạt ảnh Nâng cao với JavaScript Thuần.....	17
2.5.5. Cấu trúc Component và Tính năng Tương tác	17
2.6. Công nghệ ảo hóa và Quản trị hạ tầng (DevOps).....	18
2.6.1. Tổng quan về Công nghệ Ảo hóa và Trình điều phối Docker Compose	18
2.6.2. Quản lý Mạng nội bộ và Ánh xạ Cổng (Port Mapping)	18
2.6.3. Cơ chế Nạp lại mã nguồn tức thời (Hot-Reload).....	19
2.6.4. Quản trị cấu hình và Biến môi trường bảo mật	19
CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT	20
3.1. Tổng quan giải pháp (Solution Overview)	20
3.1.1. Mục tiêu của giải pháp	20
3.1.2. Mô hình tiếp cận	20
3.1.3. Các nguyên tắc cốt lõi.....	20
3.2. Kiến trúc hệ thống (System Architecture)	21
3.3. Hệ thống Đa tác tử (Multi-Agent System – AI Brain)	23
3.4. Quy trình vận hành & Luồng dữ liệu (Operational Workflow)	24
3.5. Cơ chế Bảo mật và An toàn (Security & Safety Framework).....	26
3.6. Quy trình Kiểm soát Chất lượng (Quality Assurance Methodology)	29
3.7. Công nghệ sử dụng (Technology Stack)	30
CHƯƠNG 4. TRIỂN KHAI VÀ KẾT QUẢ ĐẠT ĐƯỢC.....	32
4.1. Môi trường triển khai	32
4.1.1. Hạ tầng triển khai (Infrastructure)	32
4.1.2. Công nghệ phát triển (Stack tech)	33
4.2. Chi tiết triển khai các thành phần chính.....	34
4.2.1. API Gateway (Orchestration Service).....	34
4.2.2. NL2SQL Service (AI Pipeline)	35

4.2.3. Query Service (Execution Service)	36
4.2.4. Quản lý Lịch sử và Audit Trail	37
4.3. Kết quả đạt được và Đánh giá.....	38
4.3.1. Kết quả về mặt chức năng	38
4.3.2. Kết quả kiểm thử hiệu năng (K6 Load Test)	42
4.3.3. Kết quả kiểm thử bảo mật	42
4.4. Đánh giá chung	43
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	45
5.1. Kết luận.	45
5.2. Những hạn chế còn tồn tại (Limitations).....	46
5.3. Hướng phát triển trong tương lai (Future Work).....	47
TÀI LIỆU THAM KHẢO.....	49

CHỮ VIẾT TẮT

STT	Ký hiệu	Ý nghĩa (Tiếng Anh)	Ý nghĩa (Tiếng Việt)
1	API	Application Programming Interface	Giao diện lập trình ứng dụng
2	CI	Continuous Integration	Tích hợp liên tục
3	CD	Continuous Deployment	Triển khai liên tục
4	K8s	Kubernetes	Nền tảng điều phối container (Kubernetes)
5	DB	Database	Cơ sở dữ liệu
6	UI	User Interface	Giao diện người dùng
7	HPA	Horizontal Pod Autoscaler	Tự động mở rộng quy mô Pod theo chiều ngang
8	RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
9	PVC	Persistent Volume Claim	Yêu cầu cấp phát tài nguyên lưu trữ vĩnh viễn
10	PV	Persistent Volume	Tài nguyên lưu trữ vĩnh viễn trong cụm
11	OS	Order Service	Dịch vụ Quản lý Đơn hàng
12	IS	Inventory Service	Dịch vụ Quản lý Kho hàng
13	GHA	GitHub Actions	Công cụ tự động hóa quy trình làm việc của GitHub
14	SHR	Self-hosted Runner	Trình thực thi CI/CD tự cài đặt trên máy chủ nội bộ
15	OTel	OpenTelemetry	Bộ công cụ thu thập dữ liệu quan sát (Metrics, Logs, Traces)
16	YAML	YAML Ain't Markup Language	Ngôn ngữ đánh dấu dữ liệu dạng văn bản (cấu hình)
17	MQ	Message Queue	Hàng đợi thông điệp (sử dụng Apache Kafka)

DANH MỤC HÌNH ẢNH

Hình 2.1. RAG và các kiến trúc.....	10
Hình 2.2. Tổng quan bài toán	11
Hình 3.1. Kiến trúc hệ thống	23
Hình 3.2. Quy trình vận hành.....	26
Hình 3.3. Giao diện đăng nhập hệ thống tích hợp phương thức.....	29
Hình 4.1. Hạ tầng hệ thống	33
Hình 4.2. Giao diện người dùng (Frontend) tương tác với dịch vụ	35
Hình 4.3. Giao diện chính của ứng dụng Data Bridge hỗ trợ truy vấn	39
Hình 4.4. Hệ thống hỗ trợ hiển thị trực quan cấu trúc dữ liệu	40
Hình 4.5. Xem danh sách bảng và Xem sơ đồ liên kết	40
Hình 4.6. Quản lý tài khoản và định danh	41
Hình 4.7. Kafka UI	42

BẢNG PHÂN CÔNG

Tên các thành viên thực hiện	Nội dung công việc	Tiến độ công việc
Lê Hồng Phát	Thiết kế slide Developer Tổng hợp báo cáo các chương	100%
Trương Phú Kiệt	Developer Làm bài tập lab 2, 3, 4, 5	100%
Trà Đức Toàn	Lên plan project Developer Vẽ hình cần thiết	100%
Lê Ngọc Hiệp	Viết báo cáo chương 3, 4, 5 Tổng hợp hình ảnh	100%

TÓM TẮT ĐỀ TÀI BÁO CÁO

Hệ thống Agent SQL đã được xây dựng thành công dưới dạng một nền tảng đa tác vụ (Multi-Agent NL2SQL), cho phép chuyển đổi ngôn ngữ tự nhiên thành các truy vấn SQL chính xác và thực thi trực tiếp trên cơ sở dữ liệu Supabase. Về mặt kiến trúc, dự án đã hoàn thiện triển khai mô hình microservices với bốn dịch vụ hoạt động độc lập nhưng phối hợp chặt chẽ thông qua mạng nội bộ Docker. Cụ thể, API Gateway đóng vai trò điều phối trung tâm, NL2SQL Service đảm nhận xử lý ngôn ngữ tự nhiên thông qua các mô hình AI tiên tiến (như GPT-4o Mini và Google Gemini), Query Service phụ trách kết nối cơ sở dữ liệu và Frontend cung cấp giao diện tương tác trực quan cho người dùng. Toàn bộ kiến trúc này đã được quản lý và khởi chạy đồng bộ thông qua Docker Compose, tích hợp tính năng cập nhật mã nguồn theo thời gian thực (Hot Reload) để tối ưu hóa quá trình phát triển.

Thành tựu cốt lõi của hệ thống là luồng xử lý đa tác vụ (Multi-Agent Pipeline) đã được thiết lập vận hành trơn tru với ba tác nhân (Agent) chuyên biệt. Tác nhân Architect đã thực hiện tốt vai trò phân tích ý định người dùng và lập kế hoạch truy vấn dựa trên lược đồ cơ sở dữ liệu (Schema), trong khi tác nhân Generator chịu trách nhiệm sinh mã lệnh PostgreSQL tối ưu kèm theo diễn giải bằng ngôn ngữ tự nhiên. Đặc biệt, hệ thống đã tích hợp thành công tác nhân Validator hoạt động như một chốt chặn cuối cùng, có khả năng tự động kiểm tra tính đúng đắn, sửa các lỗi cú pháp nhỏ và đảm bảo mã SQL tuân thủ nghiêm ngặt các quy định an toàn trước khi chuyển qua bước thực thi.

Đối với khía cạnh bảo mật, hệ thống đã hiện thực hóa một lớp phòng thủ đa tầng (Security Layer) vô cùng chặt chẽ. Cơ chế kiểm duyệt đã được trang bị khả năng sử dụng biểu thức chính quy (Regex) nhằm phát hiện, làm sạch và từ chối các từ khóa nguy hiểm hoặc các đoạn mã mang tính chất tấn công tiêm nhiễm (SQL Injection). Hơn thế nữa, hệ thống quản lý dữ liệu đã thiết lập thành công cơ chế ép buộc mọi kết nối (session) ở chế độ chỉ đọc (read-only), tuyệt đối chặn các lệnh thay đổi cấu trúc bảng, đồng thời áp dụng thành công các giới hạn về thời gian thực thi (timeout tối đa 30 giây) và giới hạn số dòng dữ liệu trả về (mặc định 1000 dòng) để bảo vệ tài nguyên máy chủ.

Về mặt trải nghiệm người dùng, dự án đã hoàn thiện một giao diện tương tác Frontend cao cấp mang đậm phong cách Modern SaaS. Giao diện này được xây dựng bằng framework Next.js, nổi bật với hệ thống nền video lặp động được tối ưu hiệu năng bằng JavaScript thuần (requestAnimationFrame) thay vì CSS tĩnh. Sự kết hợp giữa hiệu ứng kính mờ (Glassmorphism), hệ thống kiểu chữ (typography) đạt độ chính xác đến từng pixel, cùng với các vùng nhập liệu đa chức năng hỗ trợ đếm tín dụng (credits) và hiển thị công cụ AI đang sử dụng đã mang lại một không gian làm việc chuyên nghiệp, giúp người dùng tra cứu dữ liệu một cách dễ dàng và mượt mà nhất.

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Tính cấp thiết của đề tài.

Trong môi trường kinh doanh hiện đại, việc khai thác thông tin từ các hệ cơ sở dữ liệu (như Supabase Postgres) để đưa ra quyết định nhanh chóng là nhu cầu thiết yếu của các doanh nghiệp. Tuy nhiên, để truy vấn được số liệu, người dùng cần có kỹ năng viết các câu lệnh SQL. Điều này tạo ra rào cản lớn đối với nhóm người dùng nghiệp vụ và các nhà quản lý, khiến họ phải phụ thuộc nhiều vào đội ngũ kỹ sư hoặc nhà phân tích dữ liệu, dẫn đến sự chậm trễ trong quá trình ra quyết định.

Sự ra đời của các Mô hình ngôn ngữ lớn (LLM) đã mang đến giải pháp chuyển đổi ngôn ngữ tự nhiên sang SQL (NL2SQL), giúp người dùng đặt câu hỏi và nhận kết quả một cách trực quan. Mặc dù vậy, việc áp dụng trực tiếp các LLM đơn lẻ vào hệ thống dữ liệu thực tế lại vấp phải những rủi ro lớn. Các mô hình này thường xuyên mắc phải tình trạng "ảo giác AI" (hallucination), sinh ra các truy vấn chứa các bảng hoặc cột không hề tồn tại trong schema dữ liệu. Hơn thế nữa, nếu không có cơ chế kiểm duyệt chặt chẽ, người dùng hoặc AI có thể vô tình hoặc cố ý tạo ra các câu lệnh thay đổi cấu trúc dữ liệu, gây mất an toàn thông tin và ảnh hưởng nghiêm trọng đến sự ổn định của hệ thống.

Từ bối cảnh đó, việc nghiên cứu và xây dựng một nền tảng truy vấn dữ liệu bằng ngôn ngữ tự nhiên đáp ứng được cả hai tiêu chí: thông minh trong xử lý ngôn ngữ và tuyệt đối an toàn trong thực thi, là một yêu cầu vô cùng cấp bách. Đề tài phát triển hệ thống Agent SQL với kiến trúc đa tác vụ (Multi-Agent Pipeline) được ra đời để giải quyết bài toán này. Hệ thống không chỉ tự động đối chiếu schema để ngăn chặn ảo giác AI, mà còn thiết lập một lớp bảo mật đa tầng nghiêm ngặt nhằm cường chế chế độ chỉ đọc và giới hạn tài nguyên truy vấn. Việc hoàn thành đề tài này sẽ mang lại giá trị thực tiễn to lớn, giúp doanh nghiệp rút ngắn tối đa thời gian tra cứu, loại bỏ rủi ro bảo mật do lỗi viết SQL thủ công và trao quyền tự phục vụ dữ liệu cho các phòng ban không chuyên về kỹ thuật.

1.2. Tổng quan tình hình nghiên cứu

Hiện nay, bài toán chuyển đổi ngôn ngữ tự nhiên sang câu lệnh truy vấn dữ liệu (NL2SQL) đang trải qua một bước tiến mang tính bước ngoặt nhờ sự bùng nổ của các Mô hình ngôn ngữ lớn (LLM) như GPT-4o hay hệ sinh thái Google Gemini. Các mô hình này thể hiện khả năng vượt trội trong việc thấu hiểu ngữ nghĩa, giúp người dùng dễ dàng truy xuất thông tin mà không cần am hiểu về cú pháp SQL.

Tuy nhiên, việc áp dụng trực tiếp các LLM đơn lẻ (Single-Agent) vào hệ thống cơ sở dữ liệu thực tế đang vấp phải những thách thức và rủi ro nghiêm trọng:

- Tình trạng ảo giác AI (Hallucination): Đây là một trong những nhược điểm lớn nhất của các LLM hiện tại. Khi không được định hướng rõ ràng, các

mô hình AI thường xuyên tự "tưởng tượng" ra các bảng hoặc cột không hề tồn tại trong schema thực tế, dẫn đến các truy vấn không thể thực thi.

- Lỗ hổng bảo mật và rủi ro hệ thống: Việc để một AI duy nhất vừa tạo mã vừa thực thi tạo ra một điểm mù lớn về bảo mật. Hệ thống có nguy cơ bị đánh lừa bởi các kỹ thuật tiêm nhiễm (SQL Injection) qua các mẫu ký tự đặc biệt (như ;, --, /*) hoặc vô tình tạo ra các lệnh làm thay đổi cấu trúc dữ liệu nguy hiểm như INSERT, UPDATE, DELETE, DROP.
- Khoảng cách ngữ nghĩa (Semantic Gap): Các thuật ngữ đặc thù của doanh nghiệp thường không được thể hiện rõ trên tên các bảng hay cột. Các hệ thống cũ thường không thể hiểu được các logic nghiệp vụ (business logic) này nếu không được cung cấp thêm ngữ cảnh.

Để giải quyết triệt để những vấn đề trên, xu hướng nghiên cứu trên thế giới đang chuyển dịch mạnh mẽ từ các mô hình đơn lẻ sang kiến trúc Đa tác vụ (Multi-Agent Pipeline). Trong kiến trúc này, nhiệm vụ được chia nhỏ cho các AI chuyên biệt hoạt động độc lập và tự kiểm tra chéo lẫn nhau. Diễn hình là quy trình gồm ba bước: một tác nhân chuyên phân tích ý định và trích xuất schema (Architect Agent), một tác nhân chuyên viết mã SQL dựa trên kế hoạch đã lập (Generator Agent), và đặc biệt là một tác nhân kiểm định độc lập (Validator Agent) đóng vai trò chốt chặn để tự động phát hiện, sửa lỗi và loại bỏ mọi truy vấn độc hại trước khi thực thi.

Bên cạnh đó, các nghiên cứu gần đây cũng tập trung tích hợp thêm các công nghệ nhúng (Embeddings) như text-embedding-3-large để xây dựng lớp ngữ nghĩa qua cơ sở dữ liệu Vector (như pgvector), cho phép ứng dụng kỹ thuật RAG để đưa các khái niệm nghiệp vụ vào ngữ cảnh tạo SQL. Đồng thời, các tính năng như Grounding với tìm kiếm thời gian thực cũng bắt đầu được áp dụng để giảm thiểu ảo giác AI.

Dự án Agent SQL được định hình và phát triển dựa trên việc kế thừa những bước tiến công nghệ này. Hệ thống không chỉ ứng dụng kiến trúc Multi-Agent để tối ưu hóa sự chính xác trong bài toán NL2SQL mà còn kết hợp thiết kế Microservices cùng các lớp bảo mật đa tầng (Security Layer) để củng cố chế độ truy vấn chỉ đọc (Read-only) ở tầng cơ sở dữ liệu. Đây là một hướng tiếp cận toàn diện, vừa khai thác được sức mạnh phân tích của AI, vừa đảm bảo an toàn tuyệt đối cho hệ thống dữ liệu cốt lõi.

1.3. Mục tiêu nghiên cứu

Mục tiêu cốt lõi của đề tài là nghiên cứu và xây dựng thành công Agent SQL - một hệ thống đa tác vụ (Multi-Agent NL2SQL) mạnh mẽ. Hệ thống này hướng tới việc tự động hóa quá trình chuyển đổi các yêu cầu bằng ngôn ngữ tự nhiên thành

những câu lệnh truy vấn SQL chính xác, sau đó thực thi chúng một cách an toàn trực tiếp trên hệ quản trị cơ sở dữ liệu Supabase PostgreSQL.

Mục tiêu cụ thể: Để đạt được sự toàn diện cho dự án, đề tài phân rã mục tiêu tổng quát thành các mục tiêu cụ thể trên ba phương diện chính:

- Về mặt kiến trúc và xử lý AI: Xây dựng một hệ thống phần mềm dựa trên kiến trúc Microservices hiện đại (bao gồm API Gateway, NL2SQL Service, Query Service và Frontend) nhằm tối ưu hóa hiệu suất và khả năng mở rộng. Trọng tâm là phải thiết lập được luồng xử lý đa tác nhân (Multi-Agent Pipeline) với sự phối hợp của ba tác nhân: Architect Agent, Generator Agent và Validator Agent để khắc phục tình trạng ảo giác AI và đảm bảo tính chính xác của câu lệnh.
- Về mặt bảo mật và an toàn dữ liệu: Thiết lập một lớp bảo mật đa tầng (Security Layer) vững chắc. Hệ thống phải cường chế quá trình truy vấn hoàn toàn ở chế độ chỉ đọc (Read-only), nghĩa là chỉ cho phép thực thi các lệnh SELECT hoặc WITH CTE, đồng thời từ chối mọi câu lệnh làm thay đổi cấu trúc dữ liệu (như INSERT, UPDATE, DROP). Bên cạnh đó, phải bảo vệ cơ sở dữ liệu khỏi các cuộc tấn công SQL Injection thông qua việc lọc các mẫu ký tự nguy hiểm và kiểm soát chặt chẽ tài nguyên truy vấn (giới hạn thời gian timeout và số lượng dòng trả về).
- Về mặt trải nghiệm người dùng (UI/UX): Đơn giản hóa quá trình phân tích số liệu thông qua một giao diện trực quan mang phong cách Modern SaaS. Mục tiêu là giúp các nhóm người dùng nghiệp vụ (business users) không cần kỹ năng viết SQL vẫn có thể chủ động tra cứu dữ liệu, xem trước các giải thích từ AI và nhận kết quả dưới dạng bảng hoặc biểu đồ. Điều này giúp rút ngắn đáng kể thời gian ra quyết định và trao quyền tự phục vụ dữ liệu cho các phòng ban.

Mục tiêu chiến lược dài hạn: Không chỉ dừng lại ở một công cụ tra cứu tự động, đề tài đặt mục tiêu chiến lược nâng cấp Agent SQL trở thành một Nền tảng Phân tích Dữ liệu Thông minh (Agentic Data Intelligence Platform). Hệ thống hướng tới khả năng không chỉ cung cấp dữ liệu thô mà còn tự động đưa ra các thông tin chuyên sâu (Data Insights), cho phép người dùng đào sâu vấn đề thông qua hội thoại đa bước (Multi-turn Conversation) và tích hợp các lớp ngữ nghĩa (Semantic Layer) để AI thấu hiểu sâu sắc các thuật ngữ kinh doanh đặc thù của doanh nghiệp.

1.4. Nhiệm vụ nghiên cứu

Để hoàn thành các mục tiêu nghiên cứu đã đề ra, đề tài xác định năm nhóm nhiệm vụ trọng tâm cần thực hiện xuyên suốt quá trình phát triển dự án Agent SQL.

Nhiệm vụ đầu tiên là nghiên cứu và thiết kế kiến trúc hệ thống tổng thể theo mô hình Microservices. Cụ thể, hệ thống cần được phân tách thành 4 dịch vụ hoạt động độc lập nhằm tối ưu hóa hiệu suất và khả năng mở rộng: API Gateway làm điểm tiếp nhận và điều phối trung tâm, NL2SQL Service chuyên biệt cho việc xử lý AI, Query Service chịu trách nhiệm kết nối cơ sở dữ liệu, và Frontend cung cấp trải nghiệm tương tác. Toàn bộ hạ tầng này phải được cấu hình, quản lý và đóng gói bằng Docker và Docker Compose, tích hợp cơ chế cập nhật mã nguồn theo thời gian thực (Hot-reload) để đồng bộ hóa môi trường phát triển.

Nhiệm vụ thứ hai tập trung vào cốt lõi trí tuệ nhân tạo: xây dựng luồng xử lý ngôn ngữ tự nhiên đa tác vụ (Multi-Agent Pipeline). Quá trình này đòi hỏi việc thiết kế hệ thống Prompt chặt chẽ và tích hợp API từ các mô hình ngôn ngữ lớn để tạo ra ba tác nhân phối hợp. Architect Agent phải được thiết lập để phân tích ý định người dùng và đối chiếu chính xác với schema cơ sở dữ liệu; Generator Agent chịu trách nhiệm sinh mã lệnh PostgreSQL tối ưu kèm diễn giải; và Validator Agent hoạt động như một màng lọc cuối cùng để tự động kiểm định tính đúng đắn và sửa các lỗi cú pháp nhỏ.

Nhiệm vụ thứ ba là thiết kế và phát triển lớp bảo mật đa tầng (Security Layer) nhằm bảo vệ tuyệt đối cơ sở dữ liệu Supabase. Nhiệm vụ này yêu cầu xây dựng các bộ lọc bằng biểu thức chính quy (Regex) để nhận diện, chặn đứng các từ khóa làm thay đổi cấu trúc dữ liệu (như INSERT, UPDATE, DROP) và các mẫu mã độc có nguy cơ gây SQL Injection. Quan trọng hơn, cần cấu hình hệ thống quản lý kết nối cơ sở dữ liệu (psycopg2) để cưỡng chế mọi session luôn ở chế độ chỉ đọc (readonly=True), đồng thời áp dụng chặt chẽ các giới hạn về thời gian thực thi và số lượng dữ liệu trả về.

Nhiệm vụ thứ tư xoay quanh việc phát triển giao diện người dùng (UI/UX) đạt tiêu chuẩn của các nền tảng Modern SaaS hiện đại. Giao diện frontend cần được xây dựng bằng framework Next.js, kết hợp tính tế hiệu ứng kính mờ (Glassmorphism) và tuân thủ nghiêm ngặt hệ thống kiểu chữ (typography) đã định nghĩa. Đặc biệt, cần ứng dụng kỹ thuật xử lý JavaScript thuần túy (sử dụng hàm requestAnimationFrame) để quản lý các hiệu ứng chuyển động video nền tĩnh, thay vì phụ thuộc vào CSS thông thường, nhằm mang lại trải nghiệm mượt mà nhất cho người dùng.

Nhiệm vụ cuối cùng là thực hiện kiểm thử và đánh giá toàn diện hệ thống nhằm xác thực tính khả thi và độ ổn định. Đề tài cần triển khai hệ thống kiểm thử đa miền, bao gồm: kiểm thử tính năng giao diện từ đầu đến cuối (E2E Browser Test với Playwright), kiểm thử tích hợp các luồng API (Integration Test), và kiểm thử đo lường hiệu năng chịu tải. Các bước kiểm định này đảm bảo rằng câu lệnh SQL do AI

tạo ra không chỉ chính xác với nghiệp vụ mà hệ thống còn vận hành an toàn trong mọi kịch bản sử dụng thực tế.

1.5. Phạm vi và đối tượng nghiên cứu

Hệ thống Agent SQL được thiết kế linh hoạt nhằm đáp ứng nhu cầu khai thác dữ liệu của bốn nhóm đối tượng chính trong doanh nghiệp. Nhóm thứ nhất là những người dùng nghiệp vụ, những người không có nền tảng về ngôn ngữ truy vấn SQL nhưng lại có nhu cầu tra cứu số liệu một cách nhanh chóng bằng ngôn ngữ tự nhiên để đưa ra các quyết định kinh doanh kịp thời. Nhóm thứ hai bao gồm các nhà phân tích dữ liệu mong muốn tối ưu hóa thời gian tạo lập báo cáo từ nhiều bảng liên kết; họ cần một công cụ sinh mã tự động đi kèm khả năng xem trước câu lệnh SQL và các phần giải thích chi tiết để dễ dàng kiểm chứng. Nhóm đối tượng thứ ba là các kỹ sư dữ liệu và quản trị viên hệ thống (DBA), những người đặt sự quan tâm hàng đầu vào tính bảo mật, khả năng kiểm soát tài nguyên và việc ngăn chặn các câu lệnh nguy hiểm. Cuối cùng, hệ thống hướng tới nhóm các trưởng dự án hoặc người quản lý sản phẩm thông qua việc cung cấp công cụ lưu trữ lịch sử truy vấn, biểu đồ hóa dữ liệu và dễ dàng chia sẻ kết quả trong nội bộ nhóm.

Về phạm vi ứng dụng, đề tài tập trung xây dựng một hệ thống kiến trúc Microservices để thực thi trực tiếp các truy vấn trên hệ quản trị cơ sở dữ liệu Supabase PostgreSQL. Chức năng cốt lõi của nền tảng được giới hạn nghiêm ngặt ở các tác vụ truy xuất dữ liệu an toàn, cụ thể là chỉ hỗ trợ AI tạo và thực thi các câu lệnh SELECT hoặc cấu trúc WITH CTE. Nhằm bảo vệ máy chủ, phạm vi nghiên cứu đặc biệt chú trọng vào việc thiết lập các ranh giới tài nguyên và bảo mật. Hệ thống tự động thiết lập phiên kết nối ở chế độ read-only, chặn các dấu chấm phẩy để ngăn ngừa thực thi đa câu lệnh (multi-statements), đồng thời áp dụng cứng các quy tắc giới hạn truy vấn như thời gian chờ tối đa 30 giây và số lượng kết quả trả về không được vượt quá 1000 dòng.

Bên cạnh đó, nhằm đảm bảo tính toàn vẹn tuyệt đối cho cơ sở dữ liệu gốc, đề tài cũng xác định rõ những khía cạnh nằm ngoài phạm vi thực hiện. Hệ thống hoàn toàn không hỗ trợ việc AI tự động tạo hay thực thi bất kỳ câu lệnh nào làm thay đổi, ghi đè hoặc xóa bỏ dữ liệu cốt lõi. Hơn thế nữa, mặc dù dự án được trang bị lớp kiểm duyệt an toàn, việc quản lý phân quyền truy cập dữ liệu ở mức độ vi mô, như thiết lập Bảo mật cấp độ hàng (Row Level Security - RLS) chi tiết cho từng người dùng, sẽ không do hệ thống AI này tự động can thiệp nếu cơ sở dữ liệu gốc chưa được định cấu hình từ trước.

1.6. Phương pháp nghiên cứu

Để giải quyết trọn vẹn các bài toán đặt ra, đề tài chủ yếu áp dụng phương pháp phân tích và thiết kế hệ thống theo mô hình vi dịch vụ (Microservices) kết hợp cùng

kiến trúc Multi-Agent. Thay vì sử dụng một mô hình nguyên khối truyền thống, phương pháp tiếp cận vi dịch vụ cho phép chia tách hệ thống thành bốn phân hệ hoạt động độc lập bao gồm API Gateway, NL2SQL Service, Query Service và Frontend. Cách tiếp cận này giúp cô lập rủi ro bảo mật và tối ưu hóa khả năng mở rộng. Đặc biệt, phân hệ xử lý ngôn ngữ tự nhiên được xây dựng dựa trên phương pháp Multi-Agent Pipeline, phân chia rạch ròi quy trình thành ba bước: phân tích ý định và Architect Agent, Generator Agent và kiểm định, Validator Agent. Sự phối hợp chéo này là phương pháp cốt lõi để loại bỏ tình trạng ảo giác AI (hallucination) thường gặp ở các mô hình đơn lẻ.

Về mặt tích hợp công nghệ và phát triển, đề tài sử dụng phương pháp thực nghiệm kỹ thuật dựa trên các nền tảng trí tuệ nhân tạo tiên tiến. Quá trình sinh mã SQL được thực hiện thông qua việc gọi API từ các Mô hình ngôn ngữ lớn (LLM) như hệ sinh thái Google Gemini và GPT-4o (thông qua OpenRouter). Đề tài ứng dụng chuyên sâu các kỹ thuật Kỹ nghệ Câu lệnh (Prompt Engineering) và định dạng phản hồi chuẩn JSON để ép buộc AI tuân thủ nghiêm ngặt các quy định về cú pháp cơ sở dữ liệu. Song song đó, phương pháp quản lý hạ tầng bằng mã được áp dụng thông qua Docker và Docker Compose. Toàn bộ môi trường phát triển từ Frontend đến Backend đều được container hóa và tích hợp cơ chế nạp lại tức thời (Hot-Reload), đảm bảo tính đồng bộ cao nhất giữa môi trường phát triển cục bộ và môi trường triển khai thực tế.

Đối với khía cạnh đảm bảo an toàn và chất lượng phần mềm, đề tài áp dụng phương pháp tiếp cận bảo mật đa tầng (Multi-Layer Validation) kết hợp cùng chiến lược kiểm thử toàn diện. Phương pháp bảo mật được thiết lập bằng cách sử dụng biểu thức chính quy (Regex) để tự động nhận diện, lọc bỏ các từ khóa hoặc mẫu ký tự nguy hiểm có khả năng gây tấn công tiêm nhiễm (SQL Injection). Chốt chặn an toàn cuối cùng được thực thi bằng phương pháp cấu hình cường chế mọi phiên giao dịch với cơ sở dữ liệu PostgreSQL luôn ở chế độ chỉ đọc (Read-only). Cuối cùng, tính ổn định của nền tảng được kiểm chứng bằng phương pháp kiểm thử đa miền quy mô lớn, bao gồm: kiểm thử chức năng giao diện người dùng đầu cuối, kiểm thử luồng tích hợp (Integration Test thông qua Pytest) và đo lường khả năng chịu tải (Load Test sử dụng k6) nhằm khẳng định độ tin cậy của toàn bộ hệ thống.

1.7. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, đề tài đóng góp một mô hình thiết kế tiên tiến cho bài toán xử lý ngôn ngữ tự nhiên thành câu lệnh truy vấn (NL2SQL) thông qua việc ứng dụng chuyên sâu kiến trúc Đa tác vụ (Multi-Agent Pipeline). Thay vì phụ thuộc vào một mô hình trí tuệ nhân tạo (AI) đơn lẻ vốn dễ mắc phải tình trạng ảo giác (hallucination) và tạo ra các dữ liệu không có thực, nghiên cứu này đã chứng minh tính hiệu quả của

việc phân rã quy trình thành ba vai trò độc lập: tác nhân Architect, tác nhân Generator và tác nhân Validator. Hướng tiếp cận này cung cấp một khuôn mẫu tham khảo có giá trị cao trong việc tích hợp an toàn các Mô hình ngôn ngữ lớn (LLM) như Google Gemini hay hệ sinh thái OpenRouter với các hệ quản trị cơ sở dữ liệu quan hệ chặt chẽ, đồng thời khẳng định tính khả thi của việc đóng gói hệ thống AI vào cấu trúc Microservices để tối ưu hóa hiệu suất.

Về mặt thực tiễn đối với hoạt động doanh nghiệp, hệ thống Agent SQL mang lại giá trị to lớn trong việc dân chủ hóa khả năng tiếp cận dữ liệu. Đề tài giúp tháo gỡ rào cản kỹ thuật, trao quyền cho nhóm người dùng nghiệp vụ (business users) – những người không am hiểu cú pháp lập trình – khả năng tự chủ tra cứu, trích xuất và hình ảnh hóa dữ liệu thông qua ngôn ngữ giao tiếp hàng ngày. Giải pháp này giúp rút ngắn tối đa độ trễ từ lúc hình thành câu hỏi nghiệp vụ đến lúc nhận được kết quả phân tích, qua đó đẩy nhanh tiến độ ra quyết định và nâng cao năng lực tự phục vụ (self-service) cho các phòng ban không chuyên.

Bên cạnh đó, đối với đội ngũ kỹ sư và quản trị viên cơ sở dữ liệu (DBA), đề tài mang lại một phương thức vận hành hoàn toàn an tâm. Việc viết các câu lệnh truy vấn thủ công không chỉ tốn thời gian mà còn tiềm ẩn rủi ro phá hoại hệ thống. Hệ thống giải quyết triệt để vấn đề này nhờ áp dụng lớp Multi-Layer Validation, tự động ngăn chặn các nguy cơ tấn công SQL Injection) và cường chế mọi kết nối trực tiếp đến cơ sở dữ liệu Supabase luôn ở trạng thái chỉ đọc (Read-only). Kết quả của đề tài không chỉ tạo ra một công cụ chuyển đổi NL2SQL an toàn, mà còn thiết lập một nền móng hạ tầng vững chắc để doanh nghiệp tiếp tục phát triển hướng tới một Nền tảng Phân tích Dữ liệu Thông minh với các khả năng gợi ý nhận xét hoặc phân tích ngữ nghĩa chuyên sâu trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Mô hình ngôn ngữ lớn (LLM) và các tương tác

Trong kỷ nguyên của Trí tuệ nhân tạo sinh tạo (Generative AI), các Mô hình ngôn ngữ lớn (LLM) đóng vai trò là hạt nhân cốt lõi trong việc thấu hiểu và xử lý ngôn ngữ tự nhiên.

Việc áp dụng các mô hình này vào bài toán truy vấn cơ sở dữ liệu đòi hỏi hệ thống phải tích hợp nhiều kỹ thuật tương tác đặc thù. Điều này nhằm tối ưu hóa độ chính xác, tính an toàn và khả năng tự động hóa luồng công việc.

2.1.1. Sự tích hợp đa dạng các Mô hình ngôn ngữ lớn

Hệ thống Agent SQL được thiết kế với kiến trúc vi dịch vụ linh hoạt để không phụ thuộc vào một mô hình AI đơn lẻ, mà tận dụng sức mạnh từ nhiều nền tảng tiên tiến.

Điển hình, dự án tích hợp sâu rộng hệ sinh thái Google Gemini thông qua bộ công cụ phát triển phần mềm (SDK) google-genai mới nhất. Hệ thống hỗ trợ đa dạng các phiên bản từ Gemini 2.5 Pro, Gemini 2.5 Flash, cho đến các bản thử nghiệm tối tân như Gemini 3.1 Pro Preview và Gemini 3.1 Flash Preview.

Song song với đó, dự án ứng dụng nền tảng OpenRouter đóng vai trò như một cổng giao tiếp thống nhất, tương thích hoàn toàn với tiêu chuẩn OpenAI SDK. Thông qua cổng kết nối này, hệ thống có thể điều phối và khai thác các mô hình ngôn ngữ hàng đầu khác trên thị trường, chẳng hạn như GPT-4o Mini để xử lý cả văn bản và hình ảnh đa phương thức (Multimodal).

Bên cạnh đó, hệ thống còn mở rộng khả năng xử lý bằng cách tích hợp Groq API để ứng dụng mô hình mã nguồn mở Whisper trong việc chuyển đổi âm thanh sang văn bản (Speech-to-Text) với tốc độ cao. Đồng thời, hệ thống cũng sử dụng Tavily API làm tác nhân tìm kiếm web chuyên dụng.

2.1.2. Kỹ nghệ câu lệnh (Prompt Engineering) và Chuẩn hóa JSON

Để các mô hình ngôn ngữ lớn có thể vận hành trơn tru trong một hệ thống tự động không có sự can thiệp của con người, Prompt Engineering là yếu tố mang tính quyết định.

Do bản chất AI thường có xu hướng phản hồi bằng các đoạn hội thoại dài dòng hoặc định dạng văn bản markdown, hệ thống phải áp dụng các quy tắc kiểm soát đầu ra cực kỳ nghiêm ngặt. Trọng tâm của kỹ thuật này là ép buộc AI chỉ được phép trả về định dạng chuẩn JSON thuần túy.

Cụ thể, trong hệ thống Đa tác vụ (Multi-Agent Pipeline), tác nhân phân tích ý định (Architect Agent) được yêu cầu phản hồi một cấu trúc JSON chứa danh sách các bảng dữ liệu cần thiết và một kế hoạch truy vấn chi tiết.

Kế tiếp, Generator Agent sẽ xuất ra mã SQL PostgreSQL kèm theo phần giải thích bằng ngôn ngữ tự nhiên. Trong khi đó, Validator Agent có nhiệm vụ trả về các thông số đánh giá tính đúng đắn và an toàn cùng với đoạn mã SQL đã được sửa lỗi.

Hơn nữa, system prompts cũng chứa các quy tắc bảo mật cứng. Hệ thống tuyệt đối cấm AI sử dụng dấu chấm phẩy để ngăn ngừa lỗi đa câu lệnh, đồng thời chặn mọi lệnh thao tác dữ liệu như INSERT, UPDATE hay DROP.

2.1.3. Kỹ thuật Grounding và Cơ chế giảm thiểu Ảo giác AI (Hallucination)

Một trong những thách thức lớn nhất khi áp dụng trí tuệ nhân tạo là tình trạng ảo giác AI (hallucination). Đây là hiện tượng mô hình tự suy diễn thông tin hoặc tự tạo ra các bảng, cột dữ liệu không hề tồn tại trong hệ thống.

Để khắc phục hiện tượng này đối với các thông tin bên ngoài, dự án tích hợp tính năng tìm kiếm theo thời gian thực (Google Search Grounding). Tính năng này cho phép các mô hình Gemini kết nối trực tiếp với công cụ tìm kiếm trên Internet để truy xuất thông tin mới nhất và đối chiếu thực tế.

Kết quả trả về từ mô hình sẽ được đính kèm các metadata chứa liên kết nguồn trích dẫn. Điều này cho phép hệ thống tự động chèn các liên kết này trực tiếp vào câu trả lời để xác thực tính minh bạch của thông tin.

Đối với phạm vi cơ sở dữ liệu nội bộ, cơ chế chống ảo giác được thực thi nghiêm ngặt thông qua tác nhân kiểm định. Hệ thống yêu cầu AI phải đối chiếu mã SQL với các bảng thực tế. AI chỉ được phép sử dụng các bảng, cột đã được định nghĩa trong schema.

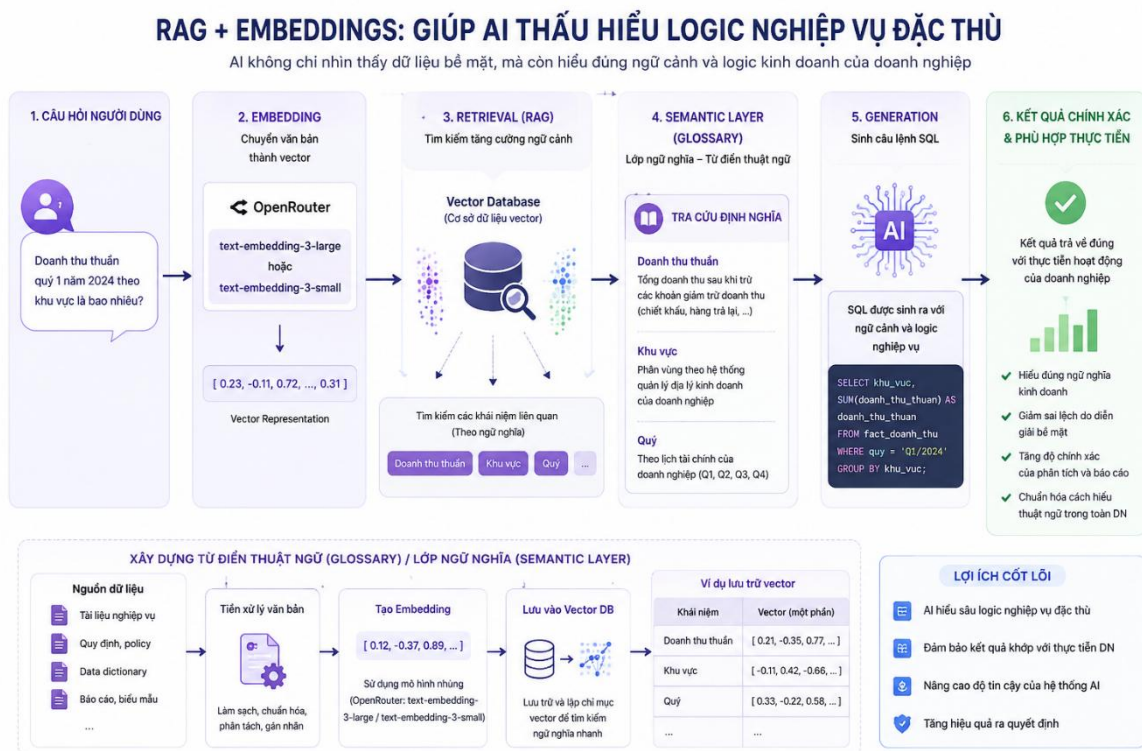
2.1.4. Lớp ngữ nghĩa (Semantic Layer) và Tìm kiếm tăng cường ngữ cảnh (RAG)

Nhằm giúp AI thấu hiểu sâu sắc các logic nghiệp vụ đặc thù của doanh nghiệp thay vì chỉ nhận diện dữ liệu ở mức độ bề mặt, hệ thống áp dụng kỹ thuật tạo lập vector nhúng (Embeddings) kết hợp cùng phương pháp Tìm kiếm tăng cường ngữ cảnh (Retrieval-Augmented Generation - RAG).

Quá trình này sử dụng các mô hình nhúng chuyên dụng như text-embedding-3-large hoặc text-embedding-3-small thông qua cổng OpenRouter. Nhờ đó, các văn bản sẽ được chuyển hóa thành các vector số học.

Các vector này sau đó được lưu trữ trong cơ sở dữ liệu hỗ trợ tìm kiếm không gian. Mục tiêu là nhằm xây dựng một Từ điển thuật ngữ (Glossary) hay Lớp ngữ nghĩa (Semantic Layer).

Nhờ đó, trước khi hệ thống sinh ra câu lệnh SQL, AI có thể tự động tra cứu định nghĩa về các khái niệm kinh doanh phức tạp. Việc đưa những logic này vào quá trình truy vấn đảm bảo kết quả trả về hoàn toàn khớp với thực tiễn hoạt động của doanh nghiệp.



Hình 2.1. RAG và các kiến trúc

2.2. Bài toán NL2SQL và Multi-Agent

Bài toán chuyển đổi ngôn ngữ tự nhiên sang câu lệnh truy vấn (NL2SQL) là thách thức cốt lõi của hệ thống, nhằm giúp những người dùng nghiệp vụ không có nền tảng kỹ thuật vẫn có thể khai thác cơ sở dữ liệu một cách dễ dàng.

Tuy nhiên, nếu chỉ sử dụng một mô hình AI duy nhất (Single-Agent) để vừa đọc hiểu yêu cầu, vừa viết mã và trực tiếp gửi đi thực thi, hệ thống sẽ phải đối mặt với rủi ro bảo mật nghiêm trọng và tình trạng "ảo giác AI" tạo ra các truy vấn không có thực.



Hình 2.2. Tổng quan bài toán

2.2.1. Tổng quan Kiến trúc Đa tác vụ (Multi-Agent Pipeline)

Để giải quyết triệt để những hạn chế của phương pháp truyền thống, dự án Agent SQL đã ứng dụng kiến trúc Đa tác vụ tại dịch vụ xử lý ngôn ngữ tự nhiên.

Kiến trúc này chia tách nhiệm vụ xử lý thành ba giai đoạn độc lập nhằm tối ưu hóa độ chính xác và đảm bảo an toàn. Hệ thống được thiết kế để duy trì luồng dữ liệu một cách nghiêm ngặt, tuyệt đối không cho phép bỏ qua bất kỳ bước nào trong chuỗi ba tác nhân: Architect, Generator và Validator.

2.2.2. Tác nhân Phân tích Ý định (Architect Agent)

Đứng ở vị trí đầu tiên trong chuỗi xử lý là Architect Agent. Tác nhân này chịu trách nhiệm tiếp nhận và phân tích sâu sắc câu hỏi bằng ngôn ngữ tự nhiên của người dùng để xác định chính xác ý định (intent) cốt lõi.

Thay vì vội vàng sinh mã ngay lập tức, Architect Agent sẽ đối chiếu yêu cầu với Lược đồ cơ sở dữ liệu (Schema) thực tế. Từ đó, nó chọn lọc và trích xuất ra một danh sách các bảng dữ liệu thực sự cần thiết cho tác vụ.

Sau khi xác định được các bảng nguồn, tác nhân này tiến hành xây dựng một Kế hoạch truy vấn (query_plan) chi tiết. Kế hoạch này phác thảo trước các logic kết nối bảng (Join) và các hàm tổng hợp (Aggregation), sau đó được đóng gói dưới định dạng JSON thuần túy để chuyển giao cho bước tiếp theo,.

2.2.3. Tác nhân Sinh mã lệnh (SQL Generator Agent)

Tiếp nối quy trình, tác nhân Generator sẽ đóng vai trò như một lập trình viên cơ sở dữ liệu. Dựa trên bản kế hoạch truy vấn chi tiết và danh sách các bảng đã được cung cấp từ Architect Agent, tác nhân này tiến hành khởi tạo câu lệnh PostgreSQL thực tế.

Để đảm bảo hiệu suất thực thi và tính dễ đọc của mã, Generator Agent được hướng dẫn sử dụng các bí danh bảng (Alias như u, o, p) giúp câu lệnh SQL trở nên ngắn gọn và tối ưu nhất. Bên cạnh việc sinh mã, một nhiệm vụ rất quan trọng khác của tác nhân này là tạo ra các đoạn giải thích (explanation) bằng ngôn ngữ tự nhiên. Điều này giúp người dùng cuối (đặc biệt là các nhà phân tích dữ liệu) dễ dàng đọc hiểu cấu trúc mã và an tâm hơn trước khi thực thi truy vấn.

2.2.4. Tác nhân Kiểm định và Sửa lỗi (Validator Agent)

Chốt chặn cuối cùng và cũng là tác nhân quan trọng nhất trong việc bảo vệ cơ sở dữ liệu là Validator Agent. Đây là lớp đánh giá độc lập nhằm kiểm tra tính đúng đắn (correctness) của câu lệnh được tạo ra so với yêu cầu ban đầu của người dùng.

Về mặt an toàn, Validator Agent thực thi các quy tắc chống "ảo giác AI" một cách khắt khe. Nó phải đối chiếu và xác minh rằng đoạn mã SQL được sinh ra chỉ sử dụng các bảng và cột thực sự tồn tại trong lược đồ dữ liệu đã cung cấp.

Tác nhân này cũng được huấn luyện để ngăn chặn tiêm nhiễm (anti-injection), tự động phát lời các yêu cầu của người dùng cố tình lách luật hệ thống. Trong trường hợp phát hiện các sai sót nhỏ về cú pháp, tác nhân này có khả năng tự động sửa lỗi và trả về đoạn mã đã hoàn thiện.

2.2.5. Các quy tắc an toàn cốt lõi trong luồng Đa tác vụ

Bên cạnh việc phân chia rõ ràng trách nhiệm của từng tác nhân, toàn bộ đường ống xử lý phải tuân thủ nghiêm ngặt một bộ quy tắc bảo mật chung, được truyền vào hệ thống thông qua các system prompts.

Quy tắc tối thượng là hợp đồng "Chỉ đọc" (SELECT-Only). Hệ thống tuyệt đối từ chối việc nói dối quy định này và cấm hoàn toàn các câu lệnh làm thay đổi cấu trúc dữ liệu như INSERT, UPDATE, DELETE, hay DROP,.

Ngoài ra, hệ thống tự động ép buộc kiểm soát tài nguyên. Nếu người dùng không chỉ định số lượng dòng muốn xem, AI được yêu cầu phải tự động gắn thêm mệnh đề LIMIT 100 để tránh tình trạng tràn dữ liệu.

Cuối cùng, AI tuyệt đối không được sử dụng dấu chấm phẩy (;) để loại bỏ nguy cơ chen đa câu lệnh (Multiple Statements). Đồng thời, chất lượng kết nối dữ liệu được kiểm soát chặt chẽ bằng cách bắt buộc sử dụng INNER JOIN hoặc LEFT JOIN với các mệnh đề ON rõ ràng, tránh hiện tượng truy vấn chéo (Cross Join) gây sập hệ thống.

2.3. Kiến trúc Microservices và Backend

Thay vì sử dụng một kiến trúc monolithic truyền thống, dự án Agent SQL được thiết kế hoàn toàn dựa trên mô hình Microservices hiện đại. Mục tiêu cốt lõi của kiến trúc này là phân tách ranh giới bảo mật rõ ràng giữa việc xử lý ngôn ngữ bằng trí tuệ nhân tạo và việc thực thi câu lệnh trực tiếp trên cơ sở dữ liệu.

Hệ thống được chia thành bốn phân hệ dịch vụ hoạt động hoàn toàn độc lập, giao tiếp với nhau thông qua mạng nội bộ HTTP của môi trường Docker, đảm bảo không gọi trực tiếp ra bên ngoài khi không cần thiết.

2.3.1. Các phân hệ vi dịch vụ lõi

Phân hệ đầu tiên là API Gateway hoạt động cố định tại cổng mạng 8000. Đây là điểm tiếp nhận trung tâm của toàn bộ hệ thống, chịu trách nhiệm điều phối luồng dữ liệu giữa giao diện người dùng và các dịch vụ xử lý phía sau, đồng thời đảm nhận nhiệm vụ quản lý lịch sử các lượt truy vấn.

Phân hệ thứ hai là Dịch vụ xử lý ngôn ngữ tự nhiên (NL2SQL Service) vận hành tại cổng 8002. Đây là nơi chứa bộ não trí tuệ nhân tạo với luồng Đa tác vụ (Multi-Agent Pipeline) sử dụng các mô hình ngôn ngữ lớn như OpenRouter hoặc Google Gemini. Nhiệm vụ duy nhất của dịch vụ này là phân tích ý định, kiểm tra lược đồ cơ sở dữ liệu và tạo ra câu lệnh SQL tối ưu.

Phân hệ thứ ba là Dịch vụ Truy vấn (Query Service) chạy ở cổng 8001. Dịch vụ này đóng vai trò như một người gác cổng, đảm nhận việc thiết lập kết nối bảo mật tới hệ quản trị Supabase PostgreSQL và thực thi các câu lệnh SQL đã được kiểm duyệt. Hệ thống áp dụng quy tắc cứng để tuyệt đối duy trì sự tách biệt, không bao giờ hợp nhất trách nhiệm giữa dịch vụ NL2SQL và dịch vụ Query.

Cuối cùng là phân hệ giao diện người dùng (Frontend) hoạt động tại cổng 3000. Phân hệ này được xây dựng bằng công nghệ Next.js, cung cấp trải nghiệm bảng điều khiển (Dashboard) trực quan và mượt mà cho người sử dụng.

2.3.2. Công nghệ phát triển API (FastAPI & Pydantic)

Về nền tảng công nghệ Backend, toàn bộ các dịch vụ hệ thống được phát triển chủ yếu bằng ngôn ngữ lập trình Python.

Dự án ứng dụng FastAPI làm framework chính để xây dựng hệ thống nhờ vào tốc độ xử lý API bất đồng bộ vượt trội. FastAPI được kết hợp chặt chẽ cùng thư viện Pydantic, đóng vai trò tự động xác thực và chuẩn hóa cấu trúc dữ liệu đầu vào một cách an toàn.

2.3.3. Quản lý kết nối cơ sở dữ liệu với Psycopg2

Để giao tiếp với cơ sở dữ liệu hệ quản trị Supabase (PostgreSQL), hệ thống ứng dụng kỹ thuật Connection Pooling (sử dụng Pooler Host ở chế độ Session Mode) để hỗ trợ ổn định giao thức kết nối IPv4 bên trong môi trường Docker.

Tại tầng Dịch vụ Truy vấn, thư viện Psycopg2 được tin dùng làm trình điều khiển cốt lõi. Quá trình duy trì kết nối được quản lý thông qua cơ chế contextmanager của Python. Điều này đảm bảo rằng mọi phiên giao dịch (connection) luôn được đóng lại một cách an toàn kể cả khi hệ thống gặp sự cố đột xuất.

Nhằm tối ưu hóa luồng dữ liệu trả về Frontend, hệ thống sử dụng công cụ RealDictCursor của thư viện Psycopg2. Công cụ này giúp định dạng kết quả truy vấn gốc thành danh sách các từ điển thay vì các mảng dữ liệu thô nguyên thủy.

Hơn thế nữa, một lớp chuyển đổi dữ liệu an toàn (Type-Safe Serialization) cũng được thiết lập. Nhờ lớp chuyển đổi này, các kiểu dữ liệu đặc thù của cơ sở dữ liệu PostgreSQL như số thập phân (Decimal) hay định dạng thời gian (DateTime) đều được tự động chuyển đổi thành dạng chuỗi văn bản hoặc chuẩn ngày tháng ISO. Kỹ thuật này giúp dữ liệu dễ dàng được đóng gói dưới định dạng JSON-serializable để truyền mượt mà lên giao diện Frontend mà không gây ra lỗi.

2.4. Hệ quản trị cơ sở dữ liệu và Cơ chế bảo mật Supabase

Trong hệ thống Agent SQL, việc quản lý và bảo vệ dữ liệu đóng vai trò sống còn. Đề tài lựa chọn hệ quản trị cơ sở dữ liệu Supabase làm nền tảng lưu trữ cốt lõi, kết hợp cùng một loạt các cơ chế bảo mật nghiêm ngặt nhằm đảm bảo an toàn tuyệt đối trước mọi rủi ro tấn công.

2.4.1. Nền tảng Supabase PostgreSQL và Cơ chế Connection Pooling

Dự án sử dụng nền tảng Supabase, một hệ sinh thái mã nguồn mở mạnh mẽ được xây dựng dựa trên lõi cơ sở dữ liệu PostgreSQL. Nền tảng này không chỉ cung cấp khả năng lưu trữ dữ liệu quan hệ truyền thống mà còn hỗ trợ các hệ thống tiên tiến, tạo tiền đề vững chắc cho việc thực thi các câu lệnh SQL phức tạp.

Để giải quyết các rào cản về giao thức mạng khi triển khai trên môi trường ảo hóa Docker, hệ thống ứng dụng kỹ thuật quản lý nhóm kết nối (Connection Pooling). Cụ thể, dự án sử dụng Pooler Host ở chế độ phiên (Session Mode), giúp duy trì tính ổn định của các giao thức kết nối IPv4 giữa các dịch vụ vi mô và máy chủ cơ sở dữ liệu.

2.4.2. Lớp bảo mật đa tầng và Chống tiêm nhiễm mã độc (SQL Injection)

Nhằm ngăn chặn triệt để các hành vi phá hoại hoặc thao tác sai lệch, hệ thống thiết lập một Lớp bảo mật đa tầng (Multi-Layer Validation) hoạt động như một màng lọc khắt khe trước khi bất kỳ truy vấn nào chạm đến cơ sở dữ liệu.

Cơ chế này sử dụng biểu thức chính quy (Regex) để thực hiện kiểm tra từ khóa (Keyword Check). Hệ thống tự động từ chối và chặn đứng mọi lệnh có khả năng làm thay đổi cấu trúc dữ liệu như các lệnh thao tác thêm, sửa, xóa, hoặc thay đổi bảng (INSERT, UPDATE, DROP, ALTER).

Bên cạnh đó, cơ chế kiểm tra mẫu mã độc (Pattern Check) được triển khai để nhận diện và loại bỏ các kỹ thuật tấn công tiêm nhiễm (SQL Injection). Hệ thống nghiêm cấm việc sử dụng dấu chấm phẩy (;) nhằm ngăn chặn nguy cơ tin tặc chen và thực thi nhiều câu lệnh cùng lúc (multiple statements). Đồng thời, các ký hiệu chú thích trong SQL (như -- hoặc /*) cũng bị vô hiệu hóa để chặn đứng các kỹ thuật đánh lừa hệ thống.

Để tăng độ chính xác của bộ lọc, hệ thống còn tích hợp chức năng bảo vệ chuỗi ký tự (String Literal Protection). Cơ chế này giúp bóc tách và phân biệt rõ ràng đâu là câu lệnh, đâu là nội dung văn bản. Nhờ đó, các từ khóa nằm bên trong chuỗi văn bản hợp lệ của người dùng sẽ không bị hệ thống nhận diện nhầm là câu lệnh độc hại.

2.4.3. Cơ chế cường chế Chỉ đọc (Read-Only) và Giới hạn tài nguyên

Chốt chặn an toàn cuối cùng và quan trọng nhất của hệ thống nằm ở cơ chế quản lý phiên kết nối của Dịch vụ Truy vấn. Thông qua trình điều khiển thư viện kết nối, hệ thống ép buộc mọi phiên làm việc với cơ sở dữ liệu phải luôn hoạt động ở trạng thái chỉ đọc (readonly=True). Ngay cả khi một câu lệnh chứa nguy cơ lọt qua được lớp kiểm duyệt văn bản, nó vẫn sẽ bị máy chủ cơ sở dữ liệu từ chối nếu mang ý đồ thay đổi thông tin.

Song song với việc bảo vệ dữ liệu, hệ thống cũng áp dụng các biện pháp bảo vệ tài nguyên máy chủ. Mọi truy vấn đều phải tuân thủ nghiêm ngặt giới hạn về thời gian thực thi (timeout) với ngưỡng tối đa là 30 giây.

Đồng thời, hệ thống cũng giới hạn số lượng kết quả trả về không được vượt quá một ngưỡng nhất định (mặc định tối đa 1000 dòng). Nếu người dùng đưa ra một truy vấn quá lớn, quy tắc này sẽ tự động ép AI hoặc hệ thống bổ sung các mệnh đề giới hạn để ngăn chặn tình trạng tràn bộ nhớ.

2.4.4. Bảo mật cấp độ hàng (RLS) và Quản trị quyền truy cập

Bên cạnh các cơ chế phòng thủ trực tiếp từ ứng dụng, hệ thống tận dụng triệt để kiến trúc bảo mật dữ liệu ở tầng cơ sở thông qua tính năng RLS (Row Level Security) của PostgreSQL.

Khi tính năng RLS được kích hoạt, nó đảm bảo rằng người dùng chỉ có thể xem được những dòng dữ liệu mà hệ thống phân quyền cho phép. Điều này có nghĩa là trong trường hợp AI tạo ra một câu lệnh truy vấn có phạm vi quét toàn bộ bảng dữ liệu, cơ chế RLS sẽ đóng vai trò như một bộ lọc ngầm, tự động cắt xén và chỉ trả về các thông tin hợp lệ với phiên đăng nhập của người dùng đó.

Hơn thế nữa, các nhà phát triển cũng phải tuân thủ các quy tắc bảo mật khắt khe khi thiết kế cơ sở dữ liệu. Điển hình là việc áp dụng các chính sách ủy quyền (security invoker) đối với các khung nhìn (views) để ngăn chặn hiện tượng khung

nhìn tự động vượt rào RLS, qua đó triệt tiêu mọi lỗ hổng bảo mật tiềm ẩn trong quá trình truy xuất thông tin.

2.5. Công nghệ phát triển Giao diện người dùng (Frontend)

Giao diện người dùng đóng vai trò là điểm chạm trực tiếp giữa hệ thống dữ liệu phức tạp và người sử dụng. Trong dự án Agent SQL, phân hệ này được thiết kế và phát triển dựa trên các tiêu chuẩn khắt khe nhất của các nền tảng dịch vụ phần mềm hiện đại nhằm mang lại trải nghiệm tra cứu dữ liệu mượt mà, trực quan và tối ưu hóa hiệu suất.

2.5.1. Nền tảng Công nghệ cốt lõi (Core Tech Stack)

Toàn bộ kiến trúc giao diện của dự án được xây dựng trên nền tảng framework Next.js kết hợp cùng thư viện React. Việc ứng dụng mô hình định tuyến App Router của Next.js giúp hệ thống tối ưu hóa tốc độ tải trang và hỗ trợ linh hoạt các phương thức kết xuất giao diện.

Để định hình phong cách trực quan một cách nhanh chóng và đồng bộ, dự án sử dụng framework TailwindCSS. Nền tảng này cho phép các nhà phát triển can thiệp sâu vào từng chi tiết hiển thị trên giao diện mà không cần phải viết các tệp CSS tùy chỉnh đồ sộ, đồng thời hỗ trợ thiết lập một hệ thống biến số thống nhất cho toàn bộ dự án. Toàn bộ phân hệ Frontend được cấu hình để hoạt động độc lập tại cổng mạng 3000, giao tiếp mượt mà với các dịch vụ Backend thông qua kiến trúc Microservices.

2.5.2. Phong cách Thiết kế Modern SaaS và Hiệu ứng Glassmorphism

Định hướng thẩm mỹ của hệ thống tuân thủ nghiêm ngặt tiêu chuẩn giao diện Modern SaaS, ưu tiên giao diện nền sáng kết hợp cùng các hiệu ứng nổi bật tinh tế.

Đặc trưng cốt lõi của phong cách này là việc áp dụng triệt để hiệu ứng kính mờ (Glassmorphism). Cụ thể, các bảng điều khiển hoặc vùng nhập liệu (như Search Input Box) được thiết lập thuộc tính làm mờ nền (backdrop-blur) kết hợp với lớp phủ tối trong suốt, ví dụ như mã màu `rgba(0,0,0,0.24)`. Kỹ thuật này giúp các khối chức năng tách biệt rõ ràng khỏi không gian nền mà vẫn duy trì được chiều sâu thị giác. Hơn thế nữa, để đảm bảo tính đồng nhất trên mọi nền tảng trình duyệt, hệ thống luôn thiết lập các tiền tố đặc thù nhằm khắc phục triệt để các lỗi mất hiệu ứng hiển thị trên trình duyệt Safari.

Bên cạnh đó, hệ thống phân cấp các nút bấm cũng được quy định rõ ràng. Nút hành động chính luôn sử dụng nền đen, chữ trắng; các nút phụ sử dụng nền trong suốt hoặc xám nhạt; và các nút mang tính chất kêu gọi nổi bật được áp dụng các dải màu xanh lá cây tương phản.

2.5.3. Hệ thống Kiểu chữ (Typography) và Bố cục không gian (Pixel-Perfect Spacing)

Dự án không sử dụng các phong chữ mặc định mà thiết lập một hệ thống Typography chuyên biệt để tạo nên bản sắc riêng. Bốn bộ phong chữ cốt lõi được hệ thống nhập liệu bao gồm Schibsted Grotesk, Inter, Noto Sans và Fustat, hỗ trợ đa dạng các độ dày từ 400 đến 700.

Việc xử lý kiểu chữ yêu cầu độ chính xác cao đến từng điểm ảnh (pixel-perfect). Chẳng hạn, các tiêu đề chính (Headline) bắt buộc phải sử dụng phong Fustat in đậm với kích thước 80px, đi kèm với cấu hình khoảng cách chữ (tracking) âm là -4.8px và triệt tiêu hoàn toàn khoảng cách dòng (line-height: none) nhằm tạo ra sự sắc nét và ấn tượng mạnh mẽ.

Về mặt không gian bố cục, hệ thống duy trì các quy chuẩn giãn cách cực kỳ nghiêm ngặt. Lề ngang tổng thể của giao diện luôn được cố định ở mức 120px. Các khoảng cách giữa các khối thành phần cũng được chuẩn hóa, ví dụ như khoảng cách từ thanh điều hướng (Navbar) đến nội dung chính là 60px, và từ tiêu đề đến khung tìm kiếm là 44px.

2.5.4. Xử lý Hoạt ảnh Nâng cao với JavaScript Thuần

Một điểm khác biệt lớn về mặt kỹ thuật giao diện của dự án là việc xử lý các hoạt ảnh phức tạp (Animation), đặc biệt là lớp video nền tĩnh. Thay vì sử dụng các thuộc tính CSS Transition thông thường vốn dễ gây ra tình trạng giật lag, hệ thống bắt buộc sử dụng JavaScript thuần túy thông qua hàm requestAnimationFrame.

Luồng xử lý này kiểm soát các hiệu ứng xuất hiện (fade-in) kéo dài 250 mili-giây ngay khi video tải xong, và hiệu ứng biến mất (fade-out) được kích hoạt chính xác khi video còn 0.55 giây trước khi kết thúc.

Để đảm bảo tính liên tục của vòng lặp (loop), hệ thống sử dụng một biến trạng thái fadingOutRef nhằm ngăn chặn việc gọi lại hiệu ứng nhiều lần gây xung đột. Mỗi khi bắt đầu một hoạt ảnh mới, mọi khung hình trước đó đều bị hủy bỏ (cancel animation frames) để hiệu ứng chuyển cảnh tiếp nối chính xác từ độ mờ (opacity) hiện tại, mang lại một trải nghiệm thị giác vô cùng mượt mà mà không gặp tình trạng giật cục.

2.5.5. Cấu trúc Component và Tính năng Tương tác

Tuân theo nguyên tắc thiết kế của React, toàn bộ mã nguồn giao diện được phân tách thành các thành phần (Component) độc lập, rạch ròi về mặt logic và chức năng.

Kiến trúc giao diện bao gồm các lớp chính:

- Thành phần VideoBackground làm nền tảng hiển thị dưới cùng
- Thành phần NavigationBar quản lý thanh điều hướng

- Thành phần HeroContent tập trung hiển thị nội dung chính.

Bên trong HeroContent, thành phần SearchInputComplex được thiết kế như một không gian làm việc thu nhỏ với cấu trúc đa tầng, bao gồm phần hiển thị tín dụng giới hạn (credits), thanh nhập liệu ngôn ngữ tự nhiên và các công cụ tiện ích bổ sung như biểu tượng đính kèm tệp hay tìm kiếm bằng giọng nói. Cách tổ chức này không chỉ giúp người dùng dễ dàng thao tác mà còn tối ưu hóa quá trình bảo trì và mở rộng tính năng của đội ngũ lập trình.

2.6. Công nghệ ảo hóa và Quản trị hạ tầng (DevOps)

Nhằm đảm bảo tính đồng bộ, sự ổn định và khả năng dễ dàng triển khai trên nhiều môi trường khác nhau, dự án Agent SQL không sử dụng phương pháp cài đặt hệ thống trực tiếp trên hệ điều hành vật lý. Thay vào đó, đề tài ứng dụng toàn diện công nghệ ảo hóa vùng chứa (Containerization) thông qua nền tảng Docker.

2.6.1. Tổng quan về Công nghệ Ảo hóa và Trình điều phối Docker Compose

Toàn bộ các phân hệ microservices của hệ thống, bao gồm API Gateway, dịch vụ xử lý AI, dịch vụ thực thi cơ sở dữ liệu (Query Service) và giao diện Frontend, đều được đóng gói thành các môi trường ảo hóa độc lập. Việc sử dụng vùng chứa giúp cô lập phần mềm cùng các thư viện phụ thuộc khỏi cấu hình hệ điều hành chủ, loại bỏ tình trạng mã nguồn chỉ chạy được trên một máy tính nhất định.

Cùng với đó, công cụ Docker Compose được sử dụng như một trình điều phối trung tâm. Thay vì phải khởi chạy từng vùng chứa một cách thủ công, hệ thống quản lý toàn bộ hệ sinh thái phần mềm thông qua tệp cấu hình duy nhất docker-compose.yml. Chỉ với một câu lệnh thực thi, toàn bộ các dịch vụ sẽ được đồng bộ hóa quá trình cài đặt, kết nối mạng và khởi động cùng lúc một cách mượt mà.

2.6.2. Quản lý Mạng nội bộ và Ánh xạ Cổng (Port Mapping)

Kiến trúc ảo hóa cho phép hệ thống thiết lập một ranh giới mạng nội bộ vô cùng chặt chẽ. Các dịch vụ bên trong dự án giao tiếp với nhau hoàn toàn qua giao thức HTTP nội bộ của hệ thống mạng Docker, từ đó ngăn chặn mọi rủi ro về việc gọi trực tiếp dữ liệu ra ngoài mạng Internet khi chưa được cấp phép.

Để người dùng và các công cụ bên ngoài có thể tương tác với hệ thống, kỹ thuật ánh xạ cổng (Port mapping) được áp dụng theo quy tắc kết nối cổng máy chủ vật lý với cổng vùng chứa. Cụ thể, hệ thống cố định Cổng giao tiếp trung tâm (API Gateway) tại số 8000 và giao diện người dùng tại cổng 3000. Các dịch vụ lõi như Query Service và NL2SQL được bảo vệ tại các cổng 8001 và 8002. Trong trường hợp máy chủ gặp sự cố do cổng mạng đã bị chiếm dụng, hệ thống cho phép kỹ sư thay đổi linh hoạt cổng ánh xạ bên ngoài mà không làm ảnh hưởng đến mã nguồn bên trong.

2.6.3. Cơ chế Nạp lại mã nguồn tức thời (Hot-Reload)

Nhằm tối ưu hóa hiệu suất làm việc của đội ngũ phát triển, môi trường ảo hóa được thiết lập đi kèm với cơ chế nạp lại mã nguồn tức thời (Hot-Reload) cho cả hệ thống Backend lẫn Frontend. Thay vì phải biên dịch lại toàn bộ hình ảnh vùng chứa mỗi khi mã nguồn có sự thay đổi, hệ thống ứng dụng kỹ thuật gắn kết ổ đĩa ảo (Volume mounting).

Theo đó, thư mục chứa mã nguồn gốc trên máy tính của nhà phát triển được ánh xạ trực tiếp vào bên trong vùng chứa. Đối với Backend, máy chủ FastAPI tự động cập nhật những thay đổi nhờ vào tham số tải lại của Uvicorn. Đối với Frontend, hệ thống thiết lập một vùng chứa định danh (named volume) độc lập dành riêng cho thư mục chứa các thư viện tải về (node_modules). Kỹ thuật này giúp loại bỏ hoàn toàn các xung đột nhân hệ điều hành giữa máy chủ vật lý và vùng chứa, đảm bảo giao diện luôn được làm mới tức thì ngay khi nhà phát triển thao tác lưu mã lệnh.

2.6.4. Quản trị cấu hình và Biến môi trường bảo mật

Về mặt bảo mật hạ tầng, toàn bộ các cấu hình nhạy cảm như khóa kết nối lập trình (API Keys), thông tin đăng nhập hệ quản trị Supabase hay định tuyến các cổng mạng đều không bao giờ được viết hardcoded vào mã nguồn phần mềm. Thay vào đó, chúng được đưa vào quản lý tập trung thông qua các tập tin biến môi trường (.env).

Trình điều phối Docker Compose sẽ làm nhiệm vụ tự động nạp các biến số này vào từng vùng chứa tương ứng thông qua thuộc tính định danh môi trường (env_file). Phương pháp luận DevOps này giúp tách biệt hoàn toàn dữ liệu nhạy cảm khỏi kho lưu trữ mã nguồn, đảm bảo tính minh bạch và độ an toàn tuyệt đối khi hệ thống Agent SQL được luân chuyển qua lại giữa các môi trường kiểm thử và môi trường triển khai thực tế.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

3.1. Tổng quan giải pháp (Solution Overview)

3.1.1. Mục tiêu của giải pháp

Hệ thống Agent SQL được định vị là một nền tảng phân tích dữ liệu tự phục vụ (Self-service Analytics Platform) với tầm nhìn chiến lược là "Bình dân hóa dữ liệu" (Democratizing data). Giải pháp được thiết kế nhằm giải quyết bài toán cốt lõi: cho phép những người dùng nghiệp vụ không có kiến thức lập trình (như Business, PM, Data Analyst) có thể dễ dàng và nhanh chóng truy vấn các cơ sở dữ liệu phức tạp thông qua giao tiếp bằng ngôn ngữ tự nhiên (NL2SQL).

3.1.2. Mô hình tiếp cận

Để hiện thực hóa mục tiêu trên mà không làm ảnh hưởng đến tính toàn vẹn của hệ thống hiện tại, Agent SQL sử dụng mô hình tiếp cận xây dựng một "Vùng đệm an toàn" (Zero-Risk Proxy) nằm giữa người dùng cuối và cơ sở dữ liệu gốc của doanh nghiệp. Mô hình này giúp giải quyết triệt để nỗi lo sập máy chủ hay rò rỉ dữ liệu từ các quản trị viên hệ thống (DBA, Data Engineer) khi cấp quyền truy cập cho người ngoài.

Hệ thống được vận hành chủ đạo bởi kiến trúc Multi-Agent (Hệ thống đa tác tử) với 3 AI Agent chuyên trách bao gồm: SQL Agent (Não bộ), QA Expert Agent (Gác cổng) và DevSecOps Expert Agent (Xương sống). Các tác tử này hoạt động phối hợp theo chuỗi dây chuyền công nghiệp nhằm phân tích dữ liệu, tự động sinh mã SQL, kiểm thử và rà soát bảo mật.

3.1.3. Các nguyên tắc cốt lõi

Hệ thống được phát triển dựa trên 3 nguyên tắc nền tảng vững chắc:

- **An toàn dữ liệu (Data Safety):** Đây là nguyên tắc tối thượng nhằm đảm bảo an toàn tuyệt đối cho cơ sở dữ liệu gốc. Giải pháp áp dụng cơ chế Read-only Proxy, sử dụng bộ lọc từ chối (Drop Filter) để lập tức báo lỗi và ngăn chặn mọi câu truy vấn chứa các từ khóa can thiệp dữ liệu như INSERT, UPDATE, DELETE, DROP. Đồng thời, hệ thống tự động ép các giới hạn tài nguyên khắt khe (Hard Limit) như chèn mệnh đề LIMIT và giới hạn thời gian (Timeout) nhằm chống tràn RAM máy chủ. Hạ tầng còn được bảo vệ bằng việc chạy Docker dưới quyền Non-root và quét lỗ hổng tự động.
- **Trải nghiệm người dùng cao cấp (Premium UX):** Hệ thống ưu tiên sự liền mạch và trực quan cho người dùng cuối. Người dùng có thể đặt câu hỏi bằng văn bản tiếng Việt/Anh và nhận về kết quả dạng bảng hoặc chuyển đổi thành biểu đồ trực quan (Line, Bar, Pie chart) chỉ với một cú nhấp chuột. Không chỉ trả về dữ liệu thô, SQL Agent còn có khả năng dịch ngược

mã SQL sang tiếng Việt để giải thích cặn kẽ logic truy vấn cho người dùng. Giao diện Frontend được xây dựng theo phong cách hiện đại Glassmorphism (Vibrant colors, blur effects) mang lại cảm giác chuyên nghiệp.

- **Khả năng mở rộng (Scalability) và tối ưu hiệu suất:** Ứng dụng kiến trúc Microservices hiện đại, hệ thống chia nhỏ thành các dịch vụ độc lập để dễ dàng tối ưu hoá hiệu suất. Hệ thống sử dụng kết hợp khéo léo cơ chế Dual-Interface: Luồng dữ liệu chat AI sử dụng xử lý bất đồng bộ qua Kafka để điều phối các tác vụ AI tốn thời gian, trong khi luồng Dashboard hiển thị sử dụng giao tiếp đồng bộ qua REST API để tận dụng xử lý song song (Parallel Refresh), giúp giảm thiểu độ trễ tối đa.

3.2. Kiến trúc hệ thống (System Architecture)

Hệ thống Agent SQL được xây dựng dựa trên nền tảng kiến trúc Microservices hiện đại, tối ưu hóa cho việc xử lý dữ liệu bằng AI và cung cấp khả năng phân tích mạnh mẽ. Việc chia nhỏ hệ thống thành các dịch vụ độc lập giúp tăng cường khả năng mở rộng, trong đó các dịch vụ giao tiếp với nhau linh hoạt thông qua sự kết hợp giữa mô hình hướng sự kiện (Event-Driven Orchestration) bằng Apache Kafka và giao tiếp đồng bộ qua REST API. Sự kết hợp khéo léo này tạo ra cơ chế "Dual-Interface", giúp hệ thống cân bằng giữa độ trễ (latency) và thông lượng (throughput) nhằm phục vụ trơn tru cả luồng phân tích AI bất đồng bộ lẫn luồng tải dữ liệu Dashboard đồng bộ. Kiến trúc tổng thể được phân tách chuyên biệt thành 5 tầng chính như sau:

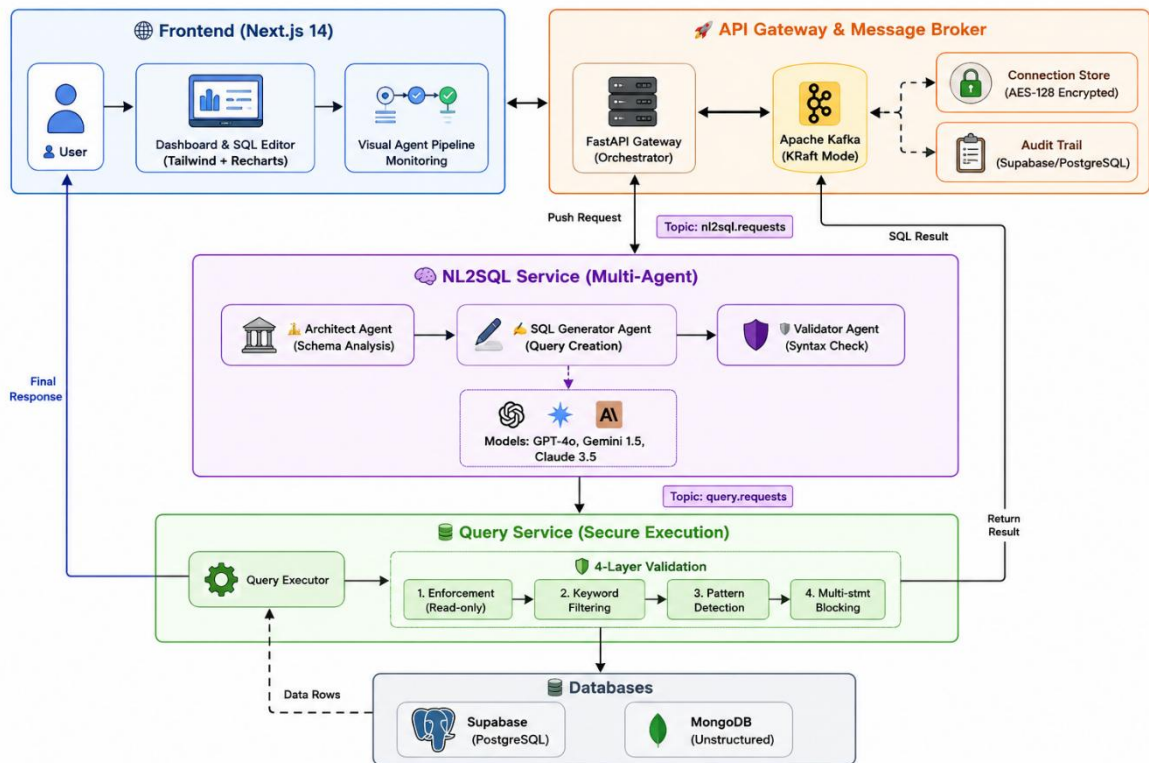
Tầng Giao diện (Frontend Layer) đóng vai trò là điểm chạm trực tiếp mang lại trải nghiệm người dùng cao cấp, được phát triển trên nền tảng Next.js 14 kết hợp phong cách thiết kế Glassmorphism hiện đại. Tầng này cung cấp không gian tương tác đa dạng bao gồm khung Chat AI để người dùng giao tiếp tự nhiên với cơ sở dữ liệu, màn hình SQL Preview để xem và chỉnh sửa câu lệnh trước khi chạy, cùng với một Dashboard động cho phép tự do cấu hình các Widget trực quan hóa dữ liệu như bảng biểu hay biểu đồ.

Tầng Điều phối (API Gateway Layer) hoạt động như một trung tâm tiếp nhận và điều hướng toàn bộ hệ thống, được xây dựng bằng framework FastAPI của Python. Nhiệm vụ cốt lõi của tầng này là quản lý định danh người dùng, lưu trữ lịch sử truy vấn và cung cấp các REST API chuẩn mực cho việc thao tác với Dashboard. Đáng chú ý, API Gateway quản lý cơ chế "Parallel Refresh" để làm mới nhiều biểu đồ trên màn hình cùng lúc qua REST, đồng thời sử dụng các mã định danh (correlation_id) để đẩy và theo dõi các yêu cầu phân tích AI phức tạp qua hệ thống Kafka một cách bất đồng bộ.

Tầng Tin nhắn (Messaging Layer) đóng vai trò là xương sống của hệ thống giao tiếp bất đồng bộ, sử dụng Apache Kafka chạy ở chế độ KRaft (không phụ thuộc vào Zookeeper). Việc ứng dụng luồng sự kiện (Event-Driven) tại tầng này đặc biệt phù hợp để giải quyết các tác vụ AI tốn nhiều thời gian. Quy trình diễn ra mượt mà khi Gateway đẩy yêu cầu câu hỏi vào một topic, các dịch vụ AI phía sau sẽ âm thầm xử lý và sau khi hoàn tất sẽ trả kết quả vào một topic phản hồi (response) để Gateway ghi nhận và chuyển tiếp lên giao diện Frontend.

Tầng Dịch vụ lõi (Core Services) là "trái tim" xử lý nghiệp vụ của toàn bộ dự án, bao gồm hai microservices chuyên trách là NL2SQL Service (Não bộ AI) và Query Service (Động cơ dữ liệu). Cụ thể, NL2SQL Service vận hành chuỗi Multi-Agent Pipeline sử dụng nền tảng CrewAI/LangChain kết hợp với LLM (như Google Gemini, OpenRouter) để phân tích cấu trúc Database, sinh mã SQL chính xác và dịch ngược logic SQL ra tiếng Việt giải thích cho người dùng. Song song đó, Query Service chịu trách nhiệm thực thi các câu lệnh trên cơ sở dữ liệu. Dịch vụ này đóng vai trò như một chốt chặn an ninh với khả năng rà soát SQL nghiêm ngặt, ép buộc giới hạn dữ liệu (Truncation) để ngăn quá tải, và linh hoạt cung cấp cả Kafka consumer lẫn REST endpoints cho hai luồng dữ liệu khác nhau.

Tầng Lưu trữ (Storage Layer) là lớp bảo vệ và lưu giữ khối lượng dữ liệu gốc của doanh nghiệp, sử dụng hệ quản trị PostgreSQL trên nền tảng Supabase. Để đảm bảo tính ổn định và khả năng phục vụ khối lượng truy vấn đồng thời cao được gửi xuống từ tầng dịch vụ lõi, kiến trúc lưu trữ được thiết lập đi kèm cơ chế Connection Pooling nhằm tối ưu hóa vòng đời kết nối và cải thiện hiệu năng toàn hệ thống.



Hình 3.1. Kiến trúc hệ thống

3.3. Hệ thống Đa tác tử (Multi-Agent System – AI Brain)

Trái tim trí tuệ nhân tạo của dự án Agent SQL nằm ở tầng dịch vụ lõi NL2SQL Service, nơi vận hành mô hình Hệ thống Đa tác tử (Multi-Agent System) sử dụng nền tảng CrewAI và LangChain kết hợp cùng các mô hình ngôn ngữ lớn (LLM). Thay vì giao phó toàn bộ quá trình xử lý ngôn ngữ tự nhiên thành một bước duy nhất dễ dẫn đến ảo giác và sai sót, giải pháp ứng dụng phương pháp phối hợp các Agent làm việc theo chuỗi (Pipeline). Phương pháp này tổ chức các tác tử AI hoạt động tuần tự như một dây chuyền công nghiệp chuyên môn hóa, nơi đầu ra của tác tử trước sẽ làm ngữ cảnh đầu vào cho tác tử sau, từ đó chia nhỏ bài toán phân tích phức tạp thành các khâu xử lý đạt độ chính xác cao nhất. Chuỗi Pipeline này được vận hành bởi 3 tác tử chuyên trách:

Khởi đầu chuỗi quy trình là tác tử **Schema Analyst Agent**. Nhiệm vụ cốt lõi của tác tử này là tiếp nhận câu hỏi bằng ngôn ngữ tự nhiên từ người dùng và phân tích chuyên sâu cấu trúc cơ sở dữ liệu (Database Schema). Schema Analyst Agent sẽ tiến hành ánh xạ ngữ nghĩa giữa ý định của người hỏi với các siêu dữ liệu thực tế (metadata), từ đó trích xuất ra chính xác những bảng và cột liên quan cần thiết để giải quyết truy vấn. Bước này đóng vai trò bản lề giúp hệ thống hiểu đúng bối cảnh dữ liệu và loại bỏ triệt để tình trạng AI tự biên soạn ra các trường dữ liệu không tồn tại.

Tiếp quản kết quả phân tích sơ đồ là **SQL Generator Agent**, đóng vai trò là "động cơ" cốt lõi của hệ thống xử lý. Dựa trên ngữ cảnh và các bảng dữ liệu đã được

ánh xạ, tác tử này thực hiện việc chuyển đổi ngôn ngữ tự nhiên sang câu lệnh SQL tối ưu. Đặc biệt, để tuân thủ nguyên tắc an toàn dữ liệu khắt khe của dự án, tác tử này bị ràng buộc bởi các quy tắc bảo mật nhằm đảm bảo chỉ sinh ra các câu lệnh truy vấn mang tính chất "chỉ đọc" (chẳng hạn như câu lệnh SELECT hoặc các biểu thức CTE), giúp loại trừ hoàn toàn các mã lệnh độc hại ngay từ khâu khởi tạo.

Mắt xích cuối cùng giúp hoàn thiện chuỗi Pipeline là **Explainer Agent**. Tác tử này được thiết kế để kết nối khoảng cách giữa hệ thống kỹ thuật khô khan và trải nghiệm của người dùng nghiệp vụ. Sau khi SQL Generator Agent hoàn tất mã lệnh, Explainer Agent sẽ tiến hành dịch ngược đoạn mã SQL này sang tiếng Việt, giải thích cặn kẽ logic trích xuất dữ liệu để người dùng có thể tự kiểm chứng xem AI đã hiểu đúng ý mình chưa. Đồng thời, tác tử này cũng phân tích kết quả dữ liệu đầu ra để đề xuất loại biểu đồ trực quan hóa (như Line, Bar, Pie chart) phù hợp nhất với ngữ cảnh của câu trả lời.

Để cả ba tác tử phối hợp nhịp nhàng và không bị chệch hướng, hệ thống áp dụng một **quy trình thiết kế Prompt (Prompt Engineering)** vô cùng khắt khe và hệ thống hóa. Các Prompt không chỉ là những câu lệnh đơn giản mà được thiết kế dưới dạng cung cấp ngữ cảnh sâu rộng, kết hợp cùng các tệp luật điều phối (AGENTS.md) để ép buộc AI tuân thủ nghiêm ngặt các tiêu chuẩn dự án và bảo mật. Trong quá trình AI vận hành, hệ thống sử dụng cơ chế theo dõi trung gian (Intermediate Tracking) nhằm ghi nhận lại từng bước suy luận của các Agent. Dữ liệu từ quá trình theo dõi này giúp các kỹ sư dễ dàng dò lỗi (debug) và liên tục tinh chỉnh Prompt. Quy trình xử lý Prompt còn được duy trì như một "Vòng lặp tri thức" (Knowledge Loop), nơi mọi quy ước mới hoặc các sai sót thường gặp đều được cập nhật ngược lại vào bộ luật của Agent, từ đó liên tục nâng cao độ chính xác của câu lệnh SQL đầu ra qua từng giai đoạn phát triển.

3.4. Quy trình vận hành & Luồng dữ liệu (Operational Workflow)

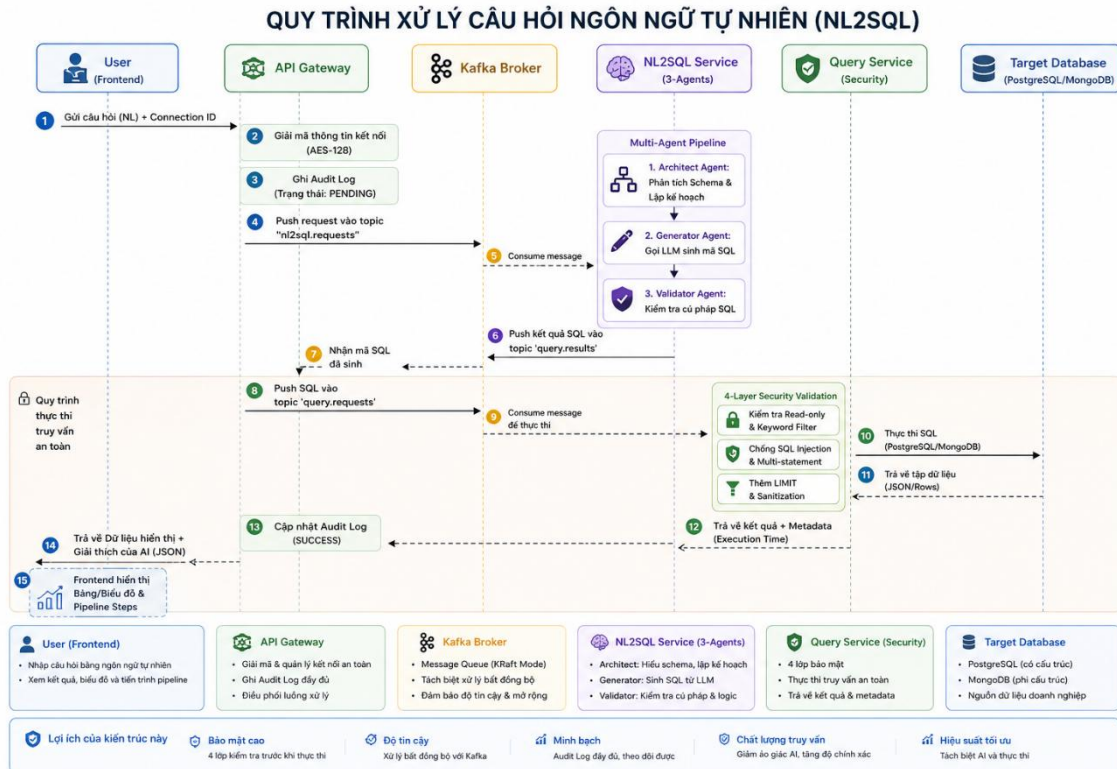
Để đảm bảo hệ thống có thể xử lý mượt mà cả các tác vụ trí tuệ nhân tạo tốn thời gian lẫn các yêu cầu truy xuất dữ liệu tức thì, Agent SQL áp dụng một cơ chế giao tiếp linh hoạt gọi là "Dual-Interface" (Giao diện kép). Khác với các kiến trúc truyền thống chỉ sử dụng một phương thức giao tiếp duy nhất, toàn bộ luồng dữ liệu của hệ thống được phân tách rõ ràng thành hai quy trình vận hành độc lập: luồng xử lý truy vấn AI bất đồng bộ và luồng hiển thị Dashboard đồng bộ.

Đối với **Luồng truy vấn AI (Asynchronous NL2SQL Flow)**, hệ thống sử dụng kiến trúc hướng sự kiện (Event-Driven) thông qua Apache Kafka để xử lý các câu hỏi bằng ngôn ngữ tự nhiên của người dùng. Quy trình bắt đầu khi người dùng nhập câu hỏi trên giao diện, API Gateway sẽ tiếp nhận yêu cầu này, gán cho nó một mã định danh duy nhất (correlation_id) và đẩy vào một topic trên Kafka. Việc giao

tiếp bất đồng bộ ở giai đoạn này là cực kỳ cần thiết vì quá trình các AI Agent phân tích ngữ nghĩa và sinh mã lệnh có thể tiêu tốn khá nhiều thời gian. Cụ thể, sau khi được Gateway điều phối, yêu cầu đi qua NL2SQL Service để xử lý và tiếp tục được chuyển tới Query Service nhằm thực thi câu lệnh truy vấn. Kết quả cuối cùng không trả về trực tiếp mà được gửi vào một topic phản hồi (response topic) trên hệ thống Kafka, nơi API Gateway đang đóng vai trò là một Consumer chờ sẵn để nhận dữ liệu, sau đó giải phóng các tác vụ (Future) và đưa kết quả lên giao diện Frontend cho người dùng.

Trái ngược với luồng AI tốn nhiều thời gian, **Luồng hiển thị Dashboard (Synchronous Dashboard Flow)** đòi hỏi tốc độ phản hồi cực kỳ nhanh để mang lại trải nghiệm người dùng tối ưu. Khi người dùng truy cập vào trang Dashboard cá nhân để xem các biểu đồ, API Gateway sẽ không đi qua bộ đệm tin nhắn Kafka mà thực hiện gọi trực tiếp đến Query Service thông qua các endpoint REST API (chẳng hạn như /execute) bằng phương thức giao tiếp đồng bộ (HTTP). Luồng dữ liệu này được thiết kế đi kèm với cơ chế Parallel Refresh (Làm mới song song), tận dụng tối đa khả năng xử lý đồng thời của FastAPI ở tầng Gateway để gửi và nhận hàng loạt yêu cầu truy vấn cùng lúc. Nhờ vậy, nhiều widget dữ liệu trên Dashboard có thể được tải và hiển thị ngay lập tức mà không gặp phải tình trạng chờ đợi theo tuần tự.

Sự ra đời của **cơ chế Dual-Interface** giải quyết bài toán cốt lõi về việc đánh đổi giữa thông lượng (throughput) và độ trễ (latency) trong kiến trúc hệ thống. Nếu chỉ sử dụng REST API cho toàn bộ các quy trình, những truy vấn AI kéo dài sẽ dễ dàng gây ra tình trạng quá thời gian chờ (timeout) làm sập hoặc nghẽn kết nối của người dùng. Ngược lại, nếu áp dụng Kafka cho mọi giao tiếp, hệ thống sẽ tự gia tăng độ trễ không cần thiết cho các tác vụ lấy dữ liệu thuần túy vốn có thể xử lý nhanh chóng. Do đó, việc chia tách luồng AI sang Kafka giúp hệ thống chịu tải tốt với khối lượng công việc phức tạp, trong khi việc giữ giao tiếp REST API cho luồng Dashboard giúp giảm thiểu tối đa độ trễ hiển thị. Query Service chính là điểm giao thoa của kiến trúc này khi cung cấp một giao diện kép, vừa đóng vai trò tiêu thụ tin nhắn từ Kafka, vừa mở sẵn các endpoint REST để phục vụ song song cả hai luồng dữ liệu nói trên.



Hình 3.2. Quy trình vận hành

3.5. Cơ chế Bảo mật và An toàn (Security & Safety Framework)

An toàn dữ liệu là nguyên tắc tối thượng và là nền tảng cốt lõi trong kiến trúc của dự án Agent SQL. Để giải quyết triệt để rủi ro sập máy chủ hay thay đổi dữ liệu ngoài ý muốn khi người dùng nghiệp vụ truy cập vào hệ thống, giải pháp này được thiết kế để hoạt động như một "Vùng đệm an toàn" (Zero-Risk Proxy) vững chắc giữa người dùng và cơ sở dữ liệu gốc. Các cơ chế bảo mật được áp dụng đồng bộ và xuyên suốt từ tầng xử lý câu lệnh cho đến mức độ hạ tầng triển khai.

Lớp bảo vệ đầu tiên và quan trọng nhất là cơ chế **Read-only Proxy** (Bộ lọc chỉ đọc) được tích hợp chặt chẽ tại tầng Backend, cụ thể là bên trong Query Service. Mặc dù các tác tử AI (như SQL Generator Agent) đã được cấp các tệp luật để chỉ sinh ra các câu lệnh an toàn dạng trích xuất (chỉ chứa SELECT hoặc biểu thức CTE), hệ thống vẫn không phó mặc hoàn toàn cho AI. Một bộ lọc từ chối (Drop Filter) tự động được thiết lập để rà soát toàn bộ các câu lệnh trước khi thực thi. Bất kỳ truy vấn nào chứa các từ khóa mang tính can thiệp dữ liệu như INSERT, UPDATE, DELETE, DROP, hay ALTER đều sẽ bị chặn đứng ngay lập tức và trả về lỗi *Unauthorized*. Khâu kiểm duyệt này cũng đồng thời chặn các câu truy vấn chứa nhiều lệnh cùng lúc (multi-statement) và ngăn ngừa nguy cơ tấn công SQL Injection, đảm bảo cơ sở dữ liệu gốc tuyệt đối không bị tác động.

Bên cạnh việc bảo vệ tính toàn vẹn của dữ liệu, Agent SQL còn áp dụng cơ chế **Hard Limit (Ép giới hạn tài nguyên)** để bảo vệ sự ổn định của hệ thống máy

chủ. Nhằm ngăn chặn tình trạng tràn RAM do người dùng vô tình yêu cầu trích xuất những bảng dữ liệu quá lớn, Backend được lập trình để tự động chèn thêm mệnh đề LIMIT (ví dụ: mặc định LIMIT 500 dòng) vào cuối mọi câu lệnh SQL. Đồng thời, quản trị viên hệ thống có quyền cấu hình thiết lập giới hạn thời gian thực thi (Timeout). Trong trường hợp một truy vấn tốn quá nhiều thời gian để xử lý, hệ thống sẽ chủ động ngắt tiến trình để giải phóng tài nguyên, đảm bảo toàn bộ hệ thống không bị treo hoặc quá tải.

Sự an toàn của Agent SQL tiếp tục được đảm bảo ở cấp độ **Hạ tầng (Infrastructure Safety)**, với sự điều phối của DevSecOps Expert Agent (Tác tử Xương sống). Hệ thống loại bỏ hoàn toàn các rủi ro bảo mật từ việc chạy ứng dụng dưới quyền cao nhất bằng cơ chế **Non-root Execution**. Mọi service đóng gói trong Docker đều bị ép buộc phải cấu hình để khởi chạy dưới quyền của một người dùng tạo riêng (appuser) thay vì quyền root. Đặc biệt, dự án vận hành một quy trình rà soát bảo mật tự động vô cùng khắt khe bằng công cụ **Trivy Scan**. DevSecOps Agent sẽ dùng Trivy để liên tục quét các image Docker nhằm dò tìm các lỗ hổng phần mềm, phát hiện cấu hình sai hoặc các nguy cơ lộ lọt mật khẩu và Secret Key. Hệ thống áp dụng một tiêu chuẩn phát hành (Definition of Done) không khoan nhượng: sẽ không có bất kỳ đoạn code nào được phép đưa lên môi trường thực tế (Production) nếu không vượt qua báo cáo kiểm thử và còn dính lỗi bảo mật nghiêm trọng (CRITICAL) từ quá trình quét Trivy này.



Create an account

Sign up to experience the power of Data Bridge

Email address

name@company.com

Password

.....

Confirm Password

.....

Sign up →

Or continue with



Continue with Google

Already have an account? [Log in](#)

Hình 3.3. Giao diện đăng nhập hệ thống tích hợp phương thức xác thực đa dạng (Email và Google OAuth) thông qua Supabase Auth

3.6. Quy trình Kiểm soát Chất lượng (Quality Assurance Methodology)

Để đảm bảo tính ổn định và độ tin cậy tuyệt đối của hệ thống trước khi đến tay người dùng, dự án Agent SQL áp dụng một quy trình kiểm soát chất lượng tự động hóa toàn diện, được vận hành chủ đạo bởi tác tử **QA Expert Agent**. Đóng vai trò như một "người gác cổng" (Gatekeeper) nghiêm ngặt, tác tử này chịu trách nhiệm quản lý toàn bộ vòng đời kiểm thử của hệ thống. Thay vì phụ thuộc vào các bước rà soát thủ công tốn thời gian, mỗi khi có một tính năng mới được phát triển, QA Expert Agent sẽ tự động kích hoạt một chuỗi quy trình gồm 5 bước khép kín: tự động sinh kịch bản kiểm thử (Test Case), kiểm thử API, kiểm thử hiệu năng, kiểm thử giao diện Frontend và cuối cùng là xuất báo cáo đánh giá tổng thể.

Quy trình tự động hóa này được thiết kế để quét qua nhiều cấp độ kiểm thử chuyên sâu nhằm đảm bảo hệ thống vận hành trơn tru ở mọi góc độ. Ở cấp độ **Kiểm thử Tích hợp API (API Integration Testing)**, tác tử ứng dụng framework Pytest kết hợp với thư viện Httpx để tự động gửi các yêu cầu giả lập đến các endpoint của hệ thống, qua đó xác thực tính chính xác trong luồng giao tiếp giữa lớp API Gateway và các dịch vụ lõi. Chuyển sang cấp độ **Kiểm thử Hiệu năng (Performance Testing)**, công cụ K6 được sử dụng để tạo ra các bài kiểm tra chịu tải (load test) lên các luồng dữ liệu trọng yếu, chẳng hạn như kiểm tra khả năng đáp ứng của hệ thống khi nhận hàng loạt yêu cầu phân tích ngôn ngữ tự nhiên hoặc khi làm mới đồng thời nhiều biểu đồ trên Dashboard. Bên cạnh đó, QA Agent cũng chạy các kịch bản kiểm thử tương tác người dùng (E2E) trên Frontend bằng công cụ Playwright.

Ở khía cạnh an toàn hệ thống, dù **Kiểm thử rà soát bảo mật (Security Scanning)** là nhiệm vụ được thực thi trực tiếp bởi DevSecOps Expert Agent, nhưng đây lại là một mảnh ghép không thể tách rời trong khâu kiểm định chất lượng cuối cùng. Hệ thống sử dụng công cụ Trivy để liên tục quét các image Docker nhằm dò tìm lỗ hổng phần mềm, cấu hình sai hoặc nguy cơ lộ lọt mật khẩu bí mật trước khi mã nguồn được phép phát hành.

Kết quả từ tất cả các cấp độ rà soát trên sẽ được tổng hợp thành một báo cáo kiểm thử hoàn chỉnh và đối chiếu với bộ **Tiêu chuẩn phát hành (Definition of Done)** vô cùng khắt khe của dự án. Một tính năng mới chỉ được coi là hoàn tất và được phép đưa lên môi trường thực tế (Production) khi thỏa mãn các điều kiện tiên quyết mang tính không khoan nhượng: toàn bộ mã nguồn phải vượt qua 100% các bài kiểm thử tích hợp API và tuyệt đối không dính bất kỳ lỗi bảo mật nào ở mức độ nghiêm trọng (CRITICAL) từ quá trình quét Trivy. Tiêu chuẩn nghiêm ngặt này chính là chốt chặn

cuối cùng giúp hệ thống Agent SQL luôn duy trì được hiệu suất cao và bảo vệ an toàn tối đa cho cơ sở dữ liệu gốc của doanh nghiệp.

3.7. Công nghệ sử dụng (Technology Stack)

Để xây dựng một nền tảng Agent SQL đáp ứng các tiêu chuẩn khắt khe về hiệu năng, khả năng mở rộng và trải nghiệm người dùng, dự án đã áp dụng một hệ sinh thái công nghệ (Technology Stack) hiện đại và đa dạng, trải dài từ các framework lập trình, nền tảng AI cho đến hạ tầng hệ thống.

Ở lớp **Giao diện người dùng (Frontend)**, hệ thống được phát triển chủ yếu bằng ngôn ngữ TypeScript trên nền tảng Next.js 14 (sử dụng App Router) kết hợp với thư viện React. Để tạo ra trải nghiệm thị giác cao cấp theo phong cách Glassmorphism (với màu sắc sống động và hiệu ứng mờ), dự án sử dụng framework TailwindCSS. Bên cạnh đó, tầng giao diện còn tích hợp các công cụ hỗ trợ mạnh mẽ như Chart.js để trực quan hóa dữ liệu thành các biểu đồ động và Monaco Editor để cung cấp khả năng hiển thị mã nguồn (Syntax Highlighting) chuyên dụng cho các đoạn code SQL.

Đối với lớp **Dịch vụ Backend (Backend Services)**, ngôn ngữ Python được lựa chọn làm nòng cốt nhờ hệ sinh thái mạnh mẽ trong lĩnh vực xử lý dữ liệu và AI. Cụ thể, framework FastAPI được sử dụng để xây dựng API Gateway và NL2SQL Service, giúp tận dụng tối đa khả năng xử lý bất đồng bộ và tốc độ phản hồi cao. Trong quá trình xử lý luồng dữ liệu, Backend còn sử dụng thư viện Pydantic để xác thực dữ liệu (data validation) và Psycopg2 tại Query Service để thiết lập các kết nối an toàn với cơ sở dữ liệu.

Trái tim trí tuệ nhân tạo của hệ thống, hay **AI Framework**, được vận hành thông qua sự kết hợp giữa CrewAI và LangChain để điều phối chuỗi Hệ thống Đa tác tử (Multi-Agent Pipeline). Khả năng phân tích ngôn ngữ tự nhiên và sinh mã lệnh được cung cấp bởi các Mô hình Ngôn ngữ Lớn (LLM) hàng đầu hiện nay, bao gồm Google Gemini và đa dạng các mô hình khác (như GPT-4o Mini) được tích hợp linh hoạt thông qua API của OpenRouter.

Về mặt **Hạ tầng và Lưu trữ (Infrastructure & Database)**, dự án sử dụng hệ quản trị cơ sở dữ liệu PostgreSQL được lưu trữ trên nền tảng Supabase. Quá trình kết nối được tối ưu hóa thông qua cơ chế Connection Pooling (Pooler Host / Session Mode) giúp hệ thống vượt qua rào cản IPv6 và duy trì hiệu năng ổn định trong môi trường Docker. Để xử lý các luồng dữ liệu bất đồng bộ giữa các tác tử AI, nền tảng Apache Kafka được triển khai theo chế độ KRaft (hoạt động độc lập không phụ thuộc vào Zookeeper) làm xương sống cho hệ thống điều phối tin nhắn. Toàn bộ các microservices của dự án đều được đóng gói bằng Docker và quản lý qua Docker

Compose, cung cấp môi trường mạng cô lập và hỗ trợ tính năng cập nhật mã nguồn tức thì (Hot Reload) cho quá trình phát triển.

Cuối cùng, hệ thống áp dụng một bộ **Công cụ Kiểm thử và Quản lý chất lượng (QA Tools)** toàn diện. Các công cụ như Pytest và Httpx được dùng để kiểm thử tích hợp API, K6 dành cho việc kiểm tra khả năng chịu tải (Performance Test), và Playwright đảm nhận kiểm thử giao diện đầu cuối (E2E). Về mặt an ninh hạ tầng, công cụ Trivy Scan được thiết lập để tự động dò tìm các lỗ hổng bảo mật trong Docker image. Ngoài ra, công cụ Mermaid cũng được sử dụng để hỗ trợ thiết kế và trực quan hóa các sơ đồ kiến trúc phức tạp của dự án.

CHƯƠNG 4. TRIỂN KHAI VÀ KẾT QUẢ ĐẠT ĐƯỢC

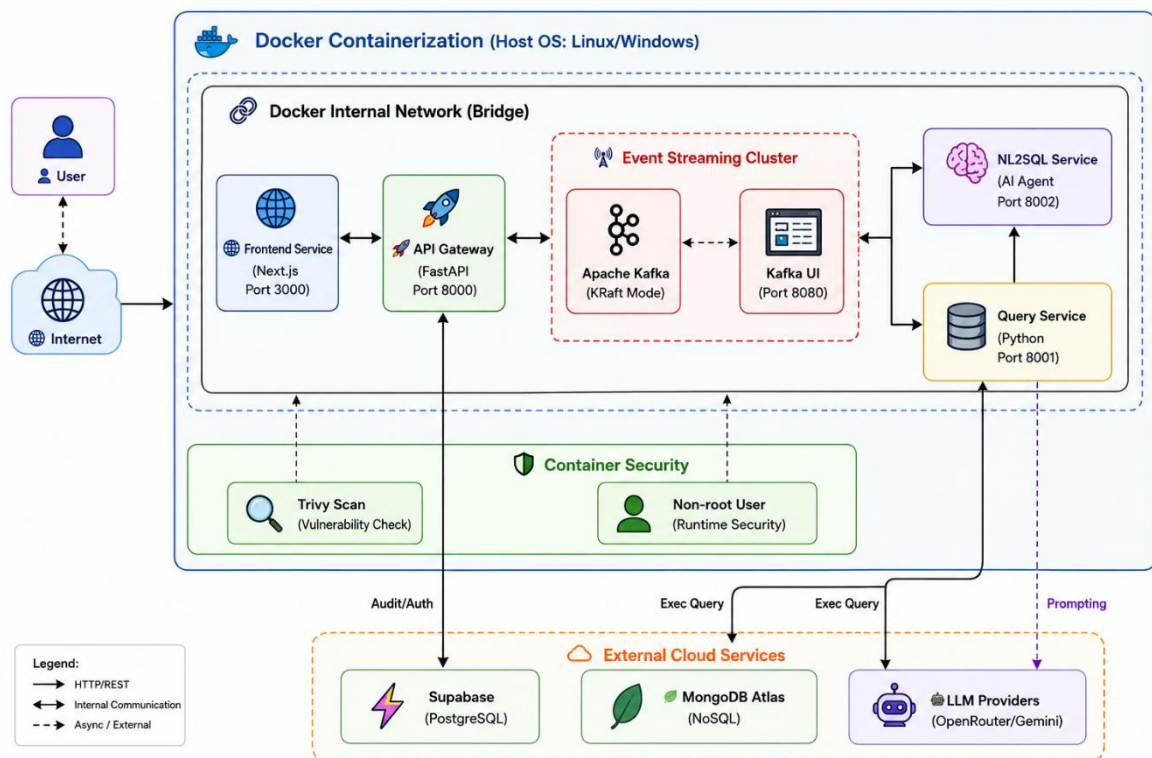
4.1. Môi trường triển khai

4.1.1. Hạ tầng triển khai (Infrastructure)

Hệ thống Agent SQL được triển khai toàn diện trên nền tảng Docker Containerization, mang lại sự đồng bộ tuyệt đối giữa các môi trường phát triển và vận hành, đồng thời đảm bảo tính đóng gói an toàn và khả năng mở rộng quy mô một cách linh hoạt. Để quản lý cụm vi dịch vụ (microservices) phức tạp này, hệ thống sử dụng công cụ điều phối docker-compose, chịu trách nhiệm vận hành và liên kết chặt chẽ 6 dịch vụ cốt lõi bao gồm: API Gateway, NL2SQL, Query Service, Frontend, Kafka và Kafka-UI.

Đóng vai trò như "trục xương sống" của toàn bộ kiến trúc là bộ điều phối tin nhắn Apache Kafka phiên bản 7.6.1, được thiết lập hoạt động theo chế độ tiên tiến KRaft. Giải pháp này đảm nhận nhiệm vụ truyền tải và điều phối các luồng tin nhắn không đồng bộ giữa các vi dịch vụ. Nhờ đó, hệ thống không chỉ duy trì được khả năng chịu tải xuất sắc trước hàng loạt truy vấn đồng thời mà còn ngăn ngừa triệt để tình trạng mất mát dữ liệu ngay cả khi xảy ra các sự cố ngoài ý muốn. Nhằm hỗ trợ công tác quản trị, hệ thống tích hợp sẵn công cụ giám sát Kafka UI, cho phép các kỹ sư dễ dàng theo dõi chi tiết trạng thái của các topics và kiểm soát luồng tin nhắn theo thời gian thực.

Tại tầng lưu trữ dữ liệu (Database Backend), hệ thống áp dụng chiến lược đa cơ sở dữ liệu (Multi-database) để tối ưu hóa hiệu suất cho từng loại dữ liệu chuyên biệt. Cụ thể, nền tảng sử dụng Supabase (dựa trên lõi PostgreSQL) làm cơ sở dữ liệu quan hệ chính, đảm nhận nhiệm vụ lưu trữ các cấu hình cốt lõi của hệ thống, thiết lập Dashboard và các siêu dữ liệu (metadata). Song song đó, nhằm đáp ứng nhu cầu mở rộng không giới hạn, hệ thống tích hợp thêm cơ sở dữ liệu NoSQL là MongoDB. Động cơ này cung cấp khả năng mở rộng lưu trữ linh hoạt và hỗ trợ truy vấn tốc độ cao đối với các khối dữ liệu phi cấu trúc, tạo nên một nền tảng lưu trữ toàn diện và vững chắc cho toàn bộ dự án.



Hình 4.1. Hạ tầng hệ thống

4.1.2. Công nghệ phát triển (Stack tech)

Hệ thống Agent SQL được xây dựng dựa trên một tổ hợp công nghệ hiện đại, tối ưu hóa cho kiến trúc vi dịch vụ (Microservices), khả năng xử lý dữ liệu tốc độ cao và tích hợp sâu rộng trí tuệ nhân tạo.

Tại tầng giao diện người dùng (Frontend), dự án ứng dụng bộ khung phát triển Next.js 14 với kiến trúc App Router tiên tiến, kết hợp cùng Tailwind CSS để xây dựng một ứng dụng web linh hoạt, mượt mà và mang đậm phong cách thiết kế hiện đại. Để hiện thực hóa mục tiêu trực quan hóa dữ liệu, hệ thống tích hợp mạnh mẽ thư viện Recharts, đóng vai trò chủ đạo trong việc chuyển hóa các tập kết quả dạng bảng khô khan thành những biểu đồ sinh động, có tính tương tác cao ngay trên bảng điều khiển (Dashboard).

Phía sau giao diện, khối dịch vụ xử lý trung tâm (Backend) được phát triển chủ đạo bằng ngôn ngữ Python 3.10+ trên nền tảng FastAPI. Việc sử dụng framework bất đồng bộ (Asynchronous framework) này giúp hệ thống đạt được hiệu năng vượt trội, dễ dàng xử lý hàng loạt truy vấn đồng thời cùng các luồng gọi API LLM mà không gây ra tình trạng nghẽn cổ chai. Song song đó, nhằm đảm bảo luồng dữ liệu vào và ra luôn chuẩn xác, thư viện Pydantic được áp dụng xuyên suốt để đảm nhiệm vai trò kiểm duyệt và xác thực (validate) dữ liệu một cách khắt khe.

Đóng vai trò "bộ não" cốt lõi của dự án là phân hệ logic trí tuệ nhân tạo (AI/LLM Logic), được vận hành thông qua nền tảng LangChain Framework. Thay vì

chỉ phụ thuộc vào một mô hình AI đơn lẻ, kiến trúc này cho phép hệ thống phối hợp linh hoạt sức mạnh của các mô hình ngôn ngữ lớn hàng đầu hiện nay như GPT-4o, Gemini 1.5 Pro và Claude 3.5 Sonnet. Sự kết hợp đa mô hình này tạo nền tảng vững chắc cho chuỗi xử lý đa tác vụ (Multi-Agent Pipeline), giúp phân tích sâu lược đồ dữ liệu và chuyển đổi ngôn ngữ tự nhiên sang câu lệnh SQL với độ chính xác cao nhất.

Cuối cùng, khía cạnh an toàn thông tin và mật mã học (Security & Cryptography) được hệ thống đặt lên hàng đầu. Để bảo vệ các thông tin hệ thống nhạy cảm, dự án ứng dụng tiêu chuẩn mã hóa Fernet (dựa trên thuật toán AES-128) nhằm mã hóa toàn bộ chuỗi kết nối cơ sở dữ liệu (Connection Strings) và mật khẩu người dùng trước khi tiến hành lưu trữ. Ở cấp độ hạ tầng, nhằm ngăn chặn các nguy cơ tấn công từ bên ngoài, hệ thống tích hợp công cụ Trivy hoạt động như một máy quét tự động. Công cụ này chịu trách nhiệm rà soát toàn diện các hình ảnh Docker (Docker Images) để phát hiện và ngăn chặn mọi lỗ hổng bảo mật hoặc cấu hình sai trước khi mã nguồn được phép đưa lên môi trường vận hành thực tế.

4.2. Chi tiết triển khai các thành phần chính

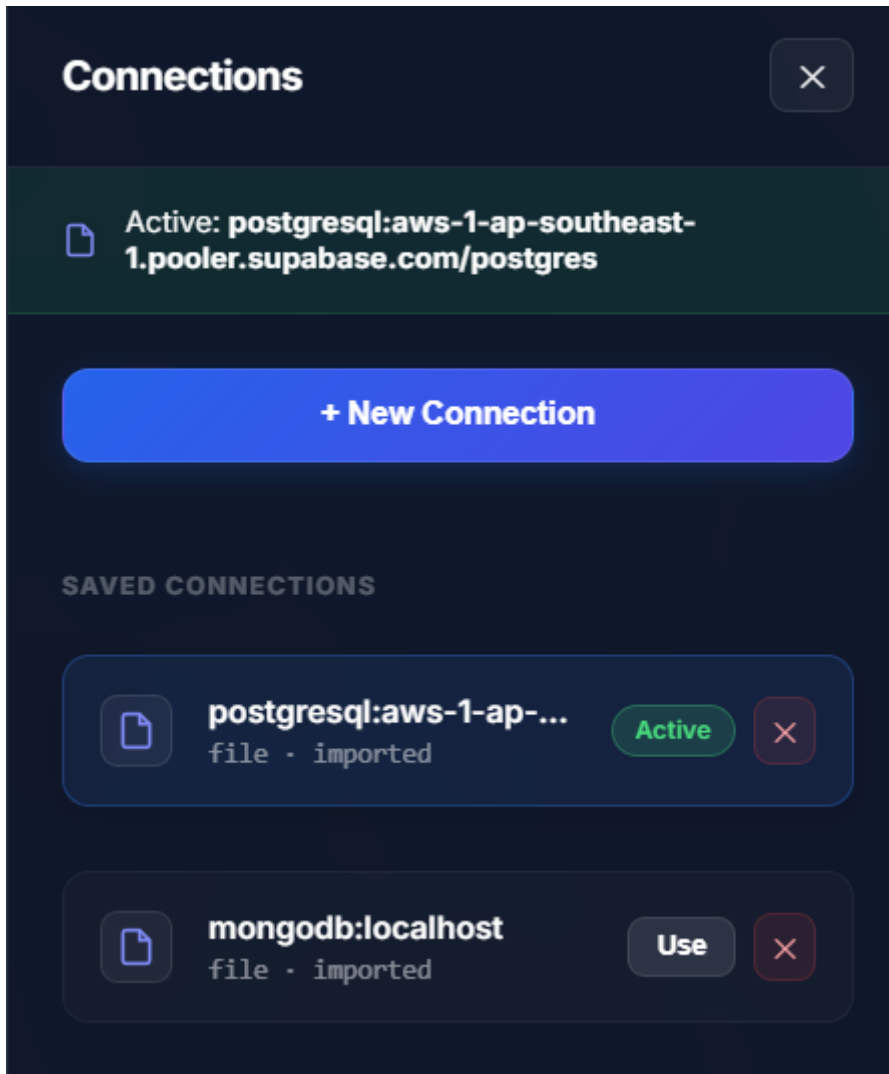
4.2.1. API Gateway (Orchestration Service)

Được xây dựng trên nền tảng FastAPI của Python, API Gateway đóng vai trò là "nhạc trưởng" (Orchestrator) chịu trách nhiệm điều phối toàn bộ các hoạt động giao tiếp và luồng xử lý sự kiện của hệ thống Agent SQL. Đây là điểm tiếp xúc đầu tiên và duy nhất, tiếp nhận mọi yêu cầu từ giao diện người dùng và định tuyến chúng đến các vi dịch vụ (microservices) tương ứng một cách chính xác.

Đối với luồng xử lý trí tuệ nhân tạo, API Gateway vận hành một cơ chế giao tiếp bất đồng bộ thông qua nền tảng Apache Kafka nhằm đảm bảo hệ thống không bị nghẽn khi xử lý các tác vụ tốn thời gian. Cụ thể, ngay khi tiếp nhận một yêu cầu truy vấn bằng ngôn ngữ tự nhiên từ người dùng, Gateway sẽ tiến hành đóng gói dữ liệu và đẩy vào chủ đề (topic) mang tên `nl2sql.requests`. Khi chuỗi tác tử AI xử lý xong và trả về mã lệnh SQL chuẩn xác, Gateway tiếp tục làm nhiệm vụ trung chuyển bằng cách nhận kết quả này và lập tức đẩy tiếp vào topic `query.requests`. Hành động này đóng vai trò như một lệnh kích hoạt, yêu cầu Query Service tiến hành thực thi câu lệnh SQL đó lên cơ sở dữ liệu mục tiêu. Xuyên suốt vòng đời của chuỗi thao tác này, hệ thống Gateway theo dõi sát sao từng tiến trình thông qua các mã định danh theo dõi (`correlation_id`), đảm bảo kết quả cuối cùng được trả về cho đúng người dùng yêu cầu.

Bên cạnh chức năng điều phối luồng tin nhắn, API Gateway còn gánh vác một trọng trách bảo mật cốt lõi là quản lý Kho lưu trữ kết nối (**Connection Store**). Nhằm bảo vệ tuyệt đối cơ sở dữ liệu gốc của doanh nghiệp, toàn bộ các thông số cấu hình kết nối nhạy cảm (như Connection Strings, địa chỉ IP hay mật khẩu người dùng) đều

không bao giờ được lưu trữ dưới dạng văn bản thô. Thay vào đó, API Gateway sẽ chịu trách nhiệm tiếp nhận, lưu trữ và giải mã các thông số này theo thời gian thực bằng một khóa bảo mật (Encryption Key) chuyên dụng dựa trên chuẩn mã hóa AES. Cơ chế này cho phép hệ thống linh hoạt kết nối với nhiều nguồn dữ liệu khác nhau mà vẫn triệt tiêu hoàn toàn rủi ro lộ lọt thông tin quản trị hệ thống.



Hình 4.2. Giao diện người dùng (Frontend) tương tác với dịch vụ Connection Store

4.2.2. NL2SQL Service (AI Pipeline)

Được ví như "bộ não" trí tuệ nhân tạo của toàn bộ dự án, Dịch vụ NL2SQL đóng vai trò then chốt trong việc thấu hiểu và chuyển hóa ý định của người dùng thành các truy vấn cơ sở dữ liệu chính xác. Để giải quyết những bài toán phân tích phức tạp, hệ thống hoàn toàn không sử dụng phương pháp ra lệnh (prompt) đơn lẻ truyền thống, mà thay vào đó triển khai một chuỗi xử lý đa tác vụ (Multi-Agent Pipeline) bao gồm 3 tác nhân (Agents) chuyên biệt hoạt động theo hình thức dây

chuyên khép kín. Kiến trúc nâng cao này được vận hành thông qua các framework như LangChain hay CrewAI, đi kèm với cơ chế theo dõi trạng thái trung gian (Intermediate Tracking) cho phép giám sát sát sao từng bước lập luận của các AI Agent.

Mở đầu cho chuỗi quy trình là Architect Agent (hay còn gọi là Schema Analyst). Tác tử này có nhiệm vụ tiếp nhận toàn bộ cấu trúc siêu dữ liệu (Metadata schema) từ hệ thống để tiến hành phân tích sâu vào cấu trúc của từng bảng cũng như các mối quan hệ (relationships) ràng buộc giữa chúng. Thông qua việc thấu hiểu ngữ cảnh nghiệp vụ, Architect Agent sẽ ánh xạ ý định bằng ngôn ngữ tự nhiên của người dùng vào lược đồ dữ liệu thực tế để lập ra một bản kế hoạch truy vấn chi tiết, qua đó loại bỏ hoàn toàn rủi ro AI tự bịa đặt (hallucination) các thông tin không tồn tại.

Ngay sau khi bản thiết kế truy vấn được hoàn thiện, luồng xử lý sẽ được tự động chuyển giao cho SQL Generator Agent. Dựa trên cấu trúc và ngữ cảnh đã được Architect Agent phân tích, tác tử này sẽ đảm nhận trọng trách cốt lõi là chuyển đổi bản kế hoạch thành các câu lệnh SQL chuẩn mực. Điểm vượt trội của SQL Generator Agent nằm ở khả năng tương thích đa nền tảng; nó có thể linh hoạt sinh ra các câu lệnh tối ưu hóa riêng biệt cho cả cơ sở dữ liệu quan hệ (PostgreSQL) lẫn cơ sở dữ liệu phi cấu trúc (MongoDB) tùy theo nguồn dữ liệu đang kết nối. Bên cạnh đó, tác tử này còn thực hiện nhiệm vụ dịch ngược câu lệnh vừa tạo sang ngôn ngữ tự nhiên tiếng Việt để giải thích logic một cách minh bạch cho người dùng cuối.

Bước cuối cùng và cũng là chốt chặn quan trọng nhất thuộc về Validator Agent. Trước khi bất kỳ câu lệnh SQL nào được phép trả về cho API Gateway để định tuyến thực thi, tác tử kiểm duyệt này sẽ tiến hành rà soát lại toàn bộ logic của câu lệnh, đảm bảo tính đúng đắn tuyệt đối về mặt cú pháp. Không chỉ dừng lại ở việc bắt lỗi ngữ pháp, Validator Agent còn đóng vai trò thực thi nghiêm ngặt các chính sách bảo mật của hệ thống bằng cách xác nhận câu lệnh tuân thủ nguyên tắc "Chỉ đọc" (Read-only), đảm bảo không có bất kỳ rủi ro nào lọt qua trước khi tương tác trực tiếp với cơ sở dữ liệu.

4.2.3. Query Service (Execution Service)

Đóng vai trò là "động cơ dữ liệu" (Data Engine) chuyên chịu trách nhiệm thực thi các truy vấn của nền tảng, Query Service được thiết kế với sự chú trọng tuyệt đối vào khả năng mở rộng kết nối và cơ chế bảo vệ an toàn. Về mặt tương thích, dịch vụ này nổi bật với khả năng hỗ trợ đa cơ sở dữ liệu, cho phép hệ thống thực thi linh hoạt các truy vấn trên cả hệ quản trị cơ sở dữ liệu quan hệ như PostgreSQL (thông qua Supabase) lẫn hệ quản trị cơ sở dữ liệu phi cấu trúc như MongoDB. Điều này giúp nền tảng dễ dàng đáp ứng mọi nhu cầu phân tích dữ liệu đa dạng của doanh nghiệp.

Nhằm thiết lập một "vùng đệm an toàn" (Zero-Risk Proxy) bảo vệ tuyệt đối cho hệ thống dữ liệu gốc, Query Service vận hành một cơ chế kiểm duyệt bảo mật đa lớp (4-Layer Validation) vô cùng khắt khe trước khi bất kỳ câu lệnh nào được chạm đến Database. Lớp bảo vệ nền tảng đầu tiên (Enforcement) thực thi nghiêm ngặt nguyên tắc "Chỉ đọc" (Read-only); theo đó, hệ thống ép buộc và chỉ cho phép chạy các câu lệnh truy xuất dữ liệu bắt đầu bằng SELECT hoặc các biểu thức bảng chung WITH (CTE). Kế tiếp, lớp lọc từ khóa (Keyword Filtering) đóng vai trò như một màng lọc từ chối (Drop Filter), tự động quét và chặn đứng mọi từ khóa có nguy cơ can thiệp hoặc phá hủy dữ liệu (DML/DDL) như DROP, DELETE, UPDATE, ALTER hay TRUNCATE. Bất kỳ truy vấn nào chứa các từ khóa này đều lập tức bị hệ thống từ chối và trả về lỗi không đủ thẩm quyền (Unauthorized).

Tiến sâu hơn vào khâu phân tích cú pháp, lớp bảo vệ thứ ba (Pattern Detection) chịu trách nhiệm rà soát chặt chẽ để phát hiện các mẫu tấn công SQL Injection tinh vi, đồng thời loại bỏ các đoạn mã chú thích (--, /*) và chặn việc gọi các hàm thực thi hệ thống trái phép. Để khép lại vòng lặp bảo mật này, lớp chốt chặn thứ tư (Multi-statement Blocking) sẽ vô hiệu hóa hoàn toàn các nỗ lực dính kèm mã độc bằng cách chặn đứng việc thực thi nhiều câu lệnh cùng lúc thông qua dấu chấm phẩy (;).

Song song với việc ngăn chặn truy vấn độc hại, dịch vụ còn tích hợp cơ chế làm sạch dữ liệu (Sanitization) kết hợp ép tài nguyên (Hard Limit) tự động. Cụ thể, sau khi qua màng lọc bảo mật, Query Service sẽ tự động làm sạch cú pháp SQL và bắt buộc nối thêm mệnh đề LIMIT vào cuối mỗi truy vấn. Quá trình can thiệp này diễn ra âm thầm ở tầng Backend, giúp kiểm soát chặt chẽ khối lượng dữ liệu trả về, từ đó loại bỏ hoàn toàn rủi ro vắt kiệt bộ nhớ (RAM) của máy chủ và đảm bảo hệ thống không bao giờ bị sập do quá tải.

4.2.4. Quản lý Lịch sử và Audit Trail

Để đảm bảo tính minh bạch, hỗ trợ đối soát dữ liệu (Audit) và tối ưu hóa hiệu năng vận hành, hệ thống Agent SQL được thiết lập một cơ chế quản lý lịch sử và lưu vết kiểm toán tự động, khép kín cho mọi thao tác truy vấn. Chức năng này không chỉ giúp người dùng cuối dễ dàng quản lý bối cảnh công việc mà còn cung cấp cho quản trị viên góc nhìn toàn cảnh về cách hệ thống đang tương tác với cơ sở dữ liệu gốc.

Về khía cạnh theo dõi và giám sát (Tracking), nền tảng vận hành một hệ thống ghi nhật ký chi tiết cho từng phiên làm việc. Mỗi khi một yêu cầu được gửi đi, hệ thống sẽ tự động lưu trữ toàn vẹn chuỗi thông tin bao gồm: câu hỏi gốc bằng ngôn ngữ tự nhiên, mã lệnh SQL chuẩn xác do AI sinh ra, phân văn bản giải thích logic của AI, và mốc thời gian thực thi. Đồng thời, chi phí Token tiêu thụ từ các mô hình ngôn ngữ lớn (LLM) cho mỗi truy vấn cũng được ghi nhận sát sao nhằm phục vụ việc kiểm soát ngân sách. Để đáp ứng tiêu chuẩn vận hành thực tế (Production-ready), dữ

liệu lịch sử này được định hướng dịch chuyển từ định dạng tệp cấu trúc tĩnh (JSON) sang lưu trữ lâu dài và bảo mật trên cơ sở dữ liệu Supabase.

Một điểm sáng về mặt kỹ thuật trong cơ chế lưu vết của nền tảng là tính năng kiểm soát phiên bản (Version Control) dành riêng cho các đoạn mã SQL. Trong quá trình sử dụng, hệ thống cho phép người dùng chuyên môn can thiệp và chỉnh sửa thủ công (Manual Edit) các câu lệnh do AI đề xuất để phù hợp hơn với nghiệp vụ. Lúc này, cơ sở dữ liệu sẽ phân tách và lưu trữ song song hai phiên bản: câu lệnh nguyên bản ban đầu (sql_original) và câu lệnh đã qua tinh chỉnh (sql_manual). Đi kèm với đó, hệ thống gắn nhãn nguồn gốc rõ ràng bằng các tham số đánh dấu (chẳng hạn như source=ai hoặc source>manual). Việc đối chiếu hai phiên bản này tạo ra một nguồn dữ liệu tham khảo quý giá, giúp đội ngũ phát triển đánh giá chính xác năng lực thực tế của AI Agent và có hướng tinh chỉnh (fine-tune) phù hợp.

Cuối cùng, nhằm hỗ trợ đánh giá và tinh chỉnh tài nguyên, hệ thống tích hợp sâu lớp ghi nhận hiệu năng (Performance Logging) vào quá trình thực thi. Mọi tương tác chạm đến cơ sở dữ liệu đều được đo lường khắt khe, tự động ghi lại thời gian thực thi tính bằng mili-giây (duration_ms) cùng với tổng số dòng dữ liệu trả về (row_count). Bộ dữ liệu kiểm toán hiệu năng này đóng vai trò then chốt giúp đội ngũ quản trị hệ thống (DBA) nhanh chóng phát hiện các truy vấn bất thường hoặc gây nghẽn mạng, đồng thời cung cấp căn cứ để phát triển các cơ chế lưu trữ bộ nhớ đệm (Cache) tối ưu trong tương lai.

4.3. Kết quả đạt được và Đánh giá

4.3.1. Kết quả về mặt chức năng

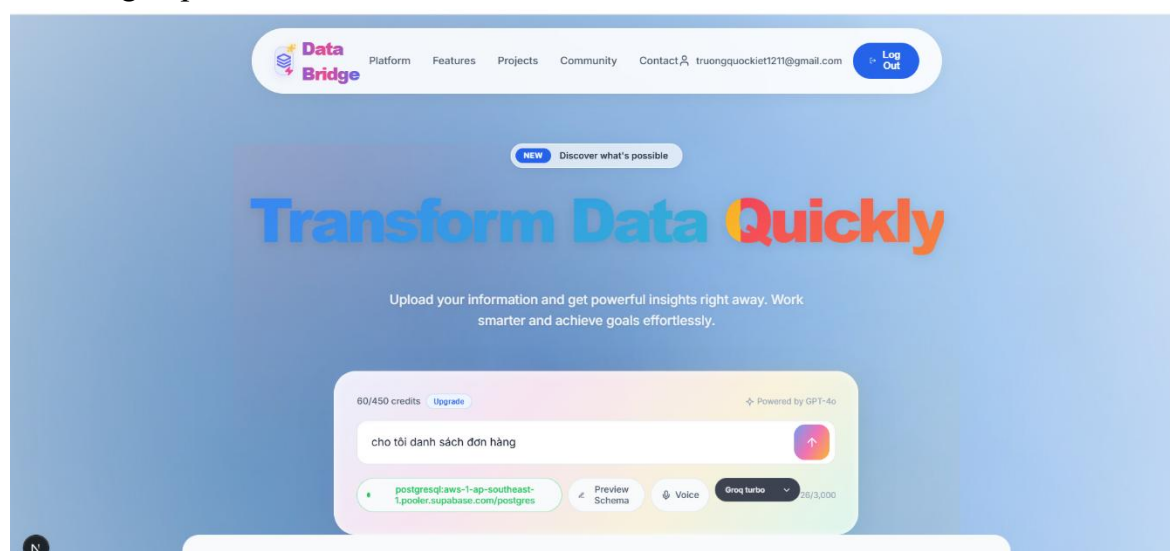
Sau quá trình triển khai và đưa vào vận hành, hệ thống Agent SQL đã đạt được những kết quả ấn tượng trên nhiều phương diện, đáp ứng toàn diện các mục tiêu đề ra ban đầu. Thành tựu nổi bật nhất phải kể đến là **độ chính xác cao** trong việc chuyển đổi ngôn ngữ tự nhiên sang câu lệnh SQL (NL2SQL). Thay vì phụ thuộc vào một luồng ra lệnh đơn lẻ, hệ thống vận hành trơn tru nhờ kiến trúc chuỗi 3 tác tử AI (Architect, SQL Generator, Validator). Sự phối hợp chuyên sâu này giúp nền tảng có khả năng xử lý xuất sắc các câu hỏi phân tích dữ liệu phức tạp, chẳng hạn như những truy vấn đòi hỏi phải kết nối chéo (Join) từ 4 đến 5 bảng dữ liệu khác nhau. Đáng chú ý, tỷ lệ sinh mã chuẩn xác của hệ thống đạt mức trên 90%, mang lại độ tin cậy cực kỳ cao cho các thao tác trích xuất thông tin nghiệp vụ.

Bên cạnh sức mạnh xử lý, nền tảng còn mang đến một trải nghiệm mượt mà và tương tác cao cho người dùng cuối. Giao diện không gian làm việc được thiết kế tối ưu với tính năng SQL Playground có hỗ trợ đánh dấu cú pháp chuyên nghiệp (SQL Syntax Highlighting). Tiện ích này trao quyền cho những người dùng có chuyên môn trực tiếp xem xét, phát hiện và sửa lỗi câu lệnh SQL do AI sinh ra ngay trên trình

soạn thảo. Mọi tình chỉnh thủ công này đều được hệ thống xử lý tức thì, cho phép cập nhật kết quả và làm mới các biểu đồ trực quan (chart) trên Dashboard gần như ngay lập tức, tạo ra một luồng công việc tự phục vụ không bị gián đoạn.

Nhằm giải quyết bài toán "hộp đen" thường gặp ở các ứng dụng trí tuệ nhân tạo, dự án đã triển khai thành công tính năng Visual Agent Pipeline (Trực quan hóa chuỗi tác tử). Thông qua cơ chế theo dõi trạng thái trung gian (Intermediate Tracking), người dùng có thể trực tiếp quan sát từng bước lập luận và xử lý của các AI Agent ngay trên giao diện hệ thống. Việc minh bạch hóa toàn bộ quá trình từ lúc phân tích lược đồ dữ liệu (Architect), viết mã (Generator) cho đến khâu kiểm duyệt an toàn (Validator) không chỉ giúp tăng cường tính minh bạch của hệ thống mà còn giúp các kỹ sư dễ dàng theo dõi, gỡ lỗi và tối ưu hóa hệ thống.

Cuối cùng, ở khía cạnh vận hành hạ tầng, hệ thống đảm bảo tiêu chuẩn quản lý kết nối an toàn tuyệt đối. Các quản trị viên và người dùng có thể linh hoạt thiết lập thêm nhiều kết nối đến các cơ sở dữ liệu khác nhau mà không phải lo lắng về rủi ro lộ lọt thông tin nhạy cảm. Kết quả này đạt được là nhờ nền tảng đã tích hợp cơ chế mã hóa chuẩn AES (cụ thể là Fernet AES-128), tự động mã hóa toàn bộ chuỗi kết nối và mật khẩu trước khi lưu trữ vào hệ thống. Lớp bảo vệ vững chắc này giúp hệ thống Agent SQL sẵn sàng đáp ứng các tiêu chuẩn bảo mật khắt khe nhất trong môi trường doanh nghiệp.

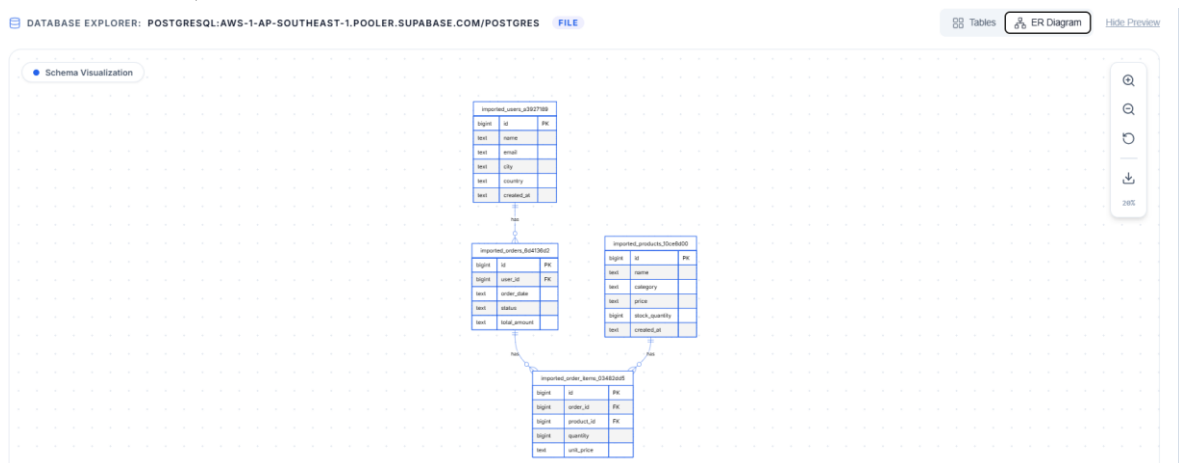


Hình 4.3. Giao diện chính của ứng dụng Data Bridge hỗ trợ truy vấn dữ liệu bằng ngôn ngữ tự nhiên

DATABASE EXPLORER: POSTGRESQL:AWS-1-AP-SOUTHEAST-1.POOLER.SUPABASE.COM/POSTGRES

Imported_order_items_0...		Imported_orders_6d4136...		Imported_products_10ce...		Imported_users_a3927189	
	Preview		Preview		Preview		Preview
~37 rows · 5 cols		~25 rows · 5 cols		~20 rows · 6 cols		~15 rows · 6 cols	
id	bigint	id	bigint	id	bigint	id	bigint
order_id	bigint	user_id	bigint	name	text	name	text
product_id	bigint	order_date	text	category	text	email	text
quantity	bigint	status	text	price	text	city	text
unit_price	text	total_amount	text	stock_quantity	bigint	country	text
				created_at	text	created_at	text

Hình 4.4. Hệ thống hỗ trợ hiển thị trực quan cấu trúc dữ liệu (Metadata visualization)



Hình 4.5. Xem danh sách bảng và Xem sơ đồ liên kết



Create an account

Sign up to experience the power of Data Bridge

Email address

 name@company.com

Password



Confirm Password



Sign up →

Or continue with

 Continue with Google

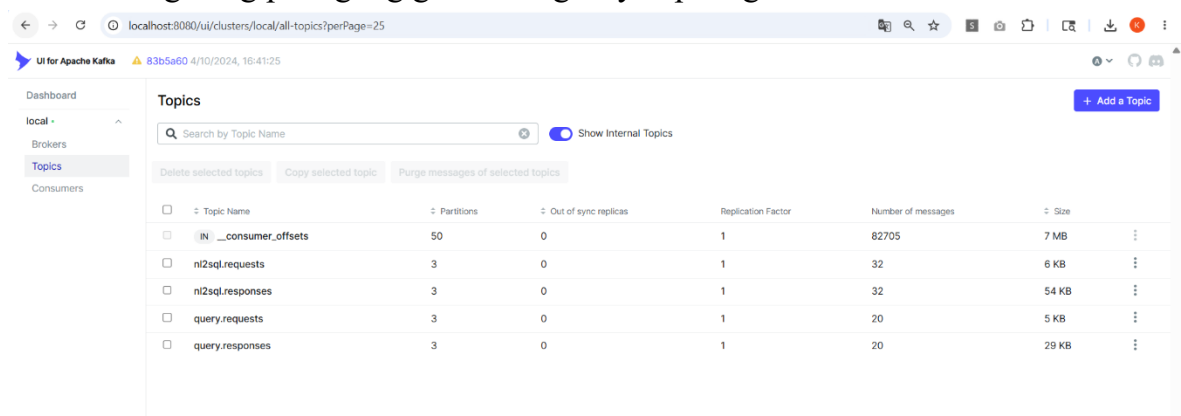
Already have an account? [Log in](#)

Hình 4.6. Quản lý tài khoản và định danh

4.3.2. Kết quả kiểm thử hiệu năng (K6 Load Test)

Để đánh giá năng lực vận hành thực tế, hệ thống Agent SQL đã trải qua các đợt kiểm thử hiệu năng khắt khe nhằm đo lường tốc độ xử lý và sự ổn định của toàn bộ kiến trúc,. Về mặt tốc độ phản hồi (Response Time), hệ thống ghi nhận thời gian xử lý trung bình chỉ ở mức 1.5 giây cho toàn bộ luồng tác vụ chuyển đổi ngôn ngữ tự nhiên sang câu lệnh SQL (NL2SQL). Điểm đặc biệt đáng chú ý là con số 1.5 giây này đã bao hàm trọn vẹn cả khoảng thời gian chờ đợi hệ thống kết nối, gọi và tiếp nhận kết quả từ các API của mô hình ngôn ngữ lớn (LLM) bên thứ ba. Đạt được thời gian phản hồi thấp như vậy trong bối cảnh phải vận hành một chuỗi đa tác tử AI (Multi-Agent Pipeline) phức tạp là một chỉ số rất ấn tượng, minh chứng cho việc hệ thống đã tối ưu hóa xuất sắc các luồng xử lý hậu cảnh để mang lại trải nghiệm tự phục vụ mượt mà và không gây gián đoạn cho người dùng cuối.

Bên cạnh tốc độ phản hồi tối ưu, sức mạnh và độ đàn hồi của hệ thống còn được thể hiện rõ nét thông qua sự bền bỉ của kiến trúc điều phối tin nhắn. Đóng vai trò là trục xương sống của toàn bộ nền tảng vi dịch vụ, hệ thống tin nhắn Apache Kafka (chạy ở chế độ KRaft) giúp nền tảng duy trì trạng thái vận hành cực kỳ ổn định, ngay cả trong những tình huống khắc nghiệt nhất khi dịch vụ NL2SQL hoặc Query Service bị quá tải và gặp độ trễ lớn,. Nhờ việc áp dụng triệt để cơ chế giao tiếp bất đồng bộ, khi các tác vụ AI đòi hỏi nhiều thời gian suy luận, Kafka sẽ ngay lập tức hoạt động như một "bộ đệm" an toàn để tiếp nhận, lưu trữ và xếp hàng các yêu cầu. Cơ chế điều phối thông minh này giúp cổng giao tiếp API Gateway luôn được giải phóng kịp thời, ngăn chặn hoàn toàn nguy cơ thắt nút cổ chai (bottleneck) và đảm bảo không có bất kỳ truy vấn nào của người dùng bị thất thoát hay báo lỗi ngay cả khi hệ thống đang phải gồng gánh lượng truy cập tăng đột biến.



Topic Name	Partitions	Out of sync replicas	Replication Factor	Number of messages	Size
_consumer_offsets	50	0	1	82705	7 MB
nl2sql.requests	3	0	1	32	6 KB
nl2sql.responses	3	0	1	32	54 KB
query.requests	3	0	1	20	5 KB
query.responses	3	0	1	20	29 KB

Hình 4.7. Kafka UI

4.3.3. Kết quả kiểm thử bảo mật

Trong quá trình phát triển và triển khai hệ thống Agent SQL, vấn đề an toàn phần mềm luôn được đặt lên hàng đầu và được thiết lập như một tiêu chuẩn "chặn

đứng" (blocker) tuyệt đối, dưới sự kiểm soát tự động khắt khe của tác tử chuyên trách DevSecOps Expert Agent. Về phương diện rà soát lỗ hổng, hệ thống đã ghi nhận kết quả kiểm thử Trivy Scan xuất sắc khi đạt chứng chỉ an toàn, hoàn toàn không tồn tại bất kỳ lỗ hổng nào ở mức độ nghiêm trọng ("Critical") bên trong các dịch vụ xử lý cốt lõi (Backend). Thành quả này đồng thời thỏa mãn trọn vẹn tiêu chuẩn chất lượng (QA Standard) mang tính bắt buộc của dự án: không có bất kỳ đoạn mã nguồn nào được phép phát hành lên môi trường vận hành thực tế (Production) nếu chưa vượt qua vòng rà soát chặt chẽ nhằm triệt tiêu mọi nguy cơ về lỗ hổng phần mềm, cấu hình sai lệch hay lộ lọt mã khóa.

Bên cạnh việc quét lỗ hổng ở trạng thái tĩnh, kiến trúc bảo mật của nền tảng còn được gia cố mạnh mẽ ở cấp độ thực thi (Runtime Security). Nhằm thiết lập một rào chắn vững chắc chống lại các nỗ lực xâm nhập hệ thống, dự án áp dụng triệt để nguyên tắc đặc quyền tối thiểu. Cụ thể, 100% các dịch vụ (services) khi được khởi chạy dưới dạng vùng chứa (Docker Container) đều bị cấm tuyệt đối việc sử dụng quyền quản trị viên (root) mặc định. Thay vào đó, toàn bộ hệ thống được cấu hình bắt buộc để chỉ hoạt động dưới quyền của một tài khoản người dùng thông thường có tên là appuser. Cơ chế thực thi không đặc quyền (Non-root Execution) này đóng vai trò như một bức tường lửa cuối cùng, giúp hạn chế tối đa rủi ro bị tấn công leo thang đặc quyền; đảm bảo rằng ngay cả khi tin tặc lợi dụng được một kẽ hở nào đó bên trong vùng chứa, chúng cũng không thể chiếm quyền kiểm soát máy chủ hạ tầng bên ngoài.

4.4. Đánh giá chung

Sau quá trình triển khai và đánh giá toàn diện, hệ thống Agent SQL đã chứng minh được tính khả thi và mang lại nhiều ưu điểm vượt trội trong việc phân tích dữ liệu tự phục vụ. Về mặt hệ thống, việc áp dụng kiến trúc vi dịch vụ (Microservices) tách biệt đã mang lại sự linh hoạt tối đa. Thiết kế này không chỉ giúp cho quá trình bảo trì diễn ra thuận lợi mà còn cho phép đội ngũ kỹ sư dễ dàng mở rộng, cập nhật hoặc thay thế từng thành phần trí tuệ nhân tạo (AI) một cách độc lập mà không làm gián đoạn đến hoạt động của toàn bộ nền tảng.

Kế tiếp, khía cạnh an toàn thông tin được hệ thống đặt lên hàng đầu và thiết lập bảo vệ xuyên suốt từ tầng lưu trữ cho đến tầng thực thi. Ở lớp lưu trữ, toàn bộ thông số kết nối cơ sở dữ liệu nhạy cảm đều được mã hóa bằng tiêu chuẩn mã hóa Fernet (AES-128) trước khi ghi vào hệ thống. Khi tiến vào tầng thực thi, các truy vấn bắt buộc phải đi qua cơ chế kiểm duyệt đa lớp (Validation) vô cùng khắt khe của Query Service nhằm chặn đứng mọi nỗ lực can thiệp dữ liệu, bảo vệ nguyên trạng hệ thống gốc. Thêm vào đó, điểm mạnh của nền tảng còn nằm ở khả năng hỗ trợ truy xuất linh hoạt trên đa dạng các nguồn dữ liệu, đáp ứng tốt từ cơ sở dữ liệu quan hệ

truyền thống như PostgreSQL cho đến các hệ quản trị phi cấu trúc hiện đại như MongoDB.

Mặc dù sở hữu nhiều ưu điểm mang tính nền tảng, hệ thống hiện tại vẫn phải đối mặt với một số hạn chế nhất định cần được giải quyết trong các phân kỳ phát triển tiếp theo. Thách thức lớn nhất hiện nay nằm ở bài toán kiểm soát ngân sách vận hành. Việc duy trì năng lực phân tích sâu sắc thông qua việc tích hợp các mô hình ngôn ngữ lớn (LLM) cao cấp như GPT-4o, Claude 3.5 Sonnet hay Gemini 1.5 Pro có thể dẫn đến sự gia tăng đột biến về chi phí gọi API. Rủi ro này đặc biệt hiện hữu khi số lượng truy vấn từ người dùng tăng mạnh hoặc khi hệ thống phải gồng gánh tính năng tự động làm mới (refresh) dữ liệu liên tục trên các Dashboard.

Để khắc phục triệt để bài toán này, định hướng phát triển sắp tới cần ưu tiên nghiên cứu và tối ưu hóa thêm cơ chế bộ nhớ đệm ngữ nghĩa (Semantic Cache), kết hợp cùng kỹ thuật nhận diện vân tay truy vấn (Query Fingerprinting). Cơ chế thông minh này sẽ cho phép hệ thống nhận biết và tái sử dụng ngay lập tức các kết quả từ những truy vấn có nội dung hoặc ngữ nghĩa tương tự đã từng được thực thi trước đó. Giải pháp này không chỉ là chìa khóa để doanh nghiệp kiểm soát và tối ưu chi phí sử dụng AI, mà còn góp phần giảm thiểu tối đa độ trễ, mang lại một trải nghiệm mượt mà và trơn vẹn hơn cho người dùng cuối.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận.

Dự án Agent SQL đã khép lại với những kết quả đáng ghi nhận, hoàn thành xuất sắc mục tiêu ban đầu là xây dựng một nền tảng phân tích dữ liệu tự phục vụ (Self-service Analytics Platform) dựa trên sức mạnh của trí tuệ nhân tạo. Hệ thống đã chứng minh được tính hiệu quả khi cho phép người dùng khai thác các cơ sở dữ liệu phức tạp thông qua giao tiếp bằng ngôn ngữ tự nhiên (NL2SQL) với độ chính xác cao, tự động chuyển đổi các câu hỏi thành truy vấn cơ sở dữ liệu,.

Một trong những thành tựu cốt lõi và quan trọng nhất của dự án là việc thiết lập thành công mô hình "Vùng đệm an toàn" (Zero-Risk Proxy). Mô hình này giải quyết triệt để bài toán đánh đổi giữa sự tiện lợi cho người dùng và tính toàn vẹn của hệ thống. Cơ sở dữ liệu gốc của doanh nghiệp được bảo vệ tuyệt đối thông qua cơ chế lọc SQL chỉ đọc (Read-only Proxy), tự động chặn đứng và từ chối mọi câu lệnh có nguy cơ can thiệp vào dữ liệu như INSERT, UPDATE, DELETE hay DROP ngay từ tầng Backend. Đồng thời, hệ thống còn áp dụng nghiêm ngặt các quy tắc giới hạn tài nguyên như tự động chèn mệnh đề LIMIT và thiết lập thời gian thực thi tối đa (Timeout) cho mỗi truy vấn, giúp ngăn chặn hoàn toàn nguy cơ quá tải hoặc sập máy chủ do các yêu cầu trích xuất dữ liệu không lồ.

Về mặt kỹ thuật, dự án đã ứng dụng thành công kiến trúc phối hợp Đa tác tử (Multi-Agent Orchestration) kết hợp với hệ thống vi dịch vụ (Microservices) hiện đại, . Thay vì xử lý nguyên khối, hệ thống hoạt động như một dây chuyền công nghiệp với các AI Agent chuyên trách (SQL Agent, QA Expert Agent và DevSecOps Expert Agent) phối hợp nhịp nhàng để phân tích cấu trúc dữ liệu, sinh mã, kiểm thử tự động và rà soát bảo mật, . Để đảm bảo hệ thống có khả năng mở rộng và chịu tải tốt, toàn bộ kiến trúc vi dịch vụ được điều phối linh hoạt thông qua hệ thống luồng sự kiện (Event-Driven) sử dụng nền tảng Apache Kafka cho các tác vụ AI tốn thời gian, song hành cùng giao tiếp đồng bộ qua REST API để đảm bảo tốc độ phản hồi tức thì cho luồng tải dữ liệu Dashboard,.

Không chỉ mạnh mẽ ở nền tảng xử lý cốt lõi, Agent SQL còn mang lại trải nghiệm tương tác giữa người và máy tối ưu thông qua một giao diện người dùng cao cấp. Được phát triển trên nền tảng Next.js hiện đại, giao diện của hệ thống áp dụng triệt để phong cách thiết kế Glassmorphism với màu sắc sống động và các hiệu ứng mờ tinh tế. Khả năng tự động chuyển đổi dữ liệu từ dạng bảng sang các biểu đồ trực quan chỉ với một thao tác cũng góp phần làm cho việc đọc hiểu dữ liệu trở nên thân thiện hơn bao giờ hết.

Nhìn chung, những kết quả kỹ thuật kể trên đã hội tụ lại để tạo ra giá trị to lớn cho doanh nghiệp, hiện thực hóa thành công tầm nhìn "Bình dân hóa dữ liệu"

(Democratizing data). Giải pháp này đã xóa bỏ hoàn toàn rào cản kỹ thuật, trao quyền tự phục vụ dữ liệu cho các phòng ban nghiệp vụ (Business, PM, Data Analyst), giúp họ tăng tốc độ ra quyết định dựa trên số liệu thực tế mà không phải chờ đợi. Đồng thời, Agent SQL cũng giải phóng đội ngũ IT và DBA khỏi những yêu cầu xuất báo cáo lặt vặt (ad-hoc), xóa bỏ nỗi lo rủi ro bảo mật hệ thống khi cấp quyền truy cập cho người ngoài, từ đó giúp họ tập trung nguồn lực cho những nhiệm vụ quản trị quan trọng hơn.

5.2. Những hạn chế còn tồn tại (Limitations)

Mặc dù hệ thống Agent SQL đã đạt được những thành tựu đáng kể trong việc hiện thực hóa tầm nhìn "bình dân hóa dữ liệu", giải pháp hiện tại vẫn còn tồn tại một số hạn chế nhất định về mặt kỹ thuật và phạm vi ứng dụng cần được khắc phục trong các phiên bản tiếp theo.

Hạn chế đầu tiên cần nhắc đến là vấn đề về độ trễ (Latency) trong quá trình xử lý truy vấn. Do trái tim của hệ thống phụ thuộc hoàn toàn vào các Mô hình Ngôn ngữ Lớn (LLM) như Google Gemini hay GPT-4o Mini để phân tích ngữ nghĩa và sinh mã SQL, thời gian phản hồi tự nhiên sẽ kéo dài hơn so với các phương pháp truy xuất truyền thống. Mặc dù kiến trúc đã ứng dụng nền tảng Apache Kafka để điều phối các tác vụ AI theo dạng bất đồng bộ nhằm chống treo hệ thống, nhưng cơ chế truyền tải tin nhắn qua Message Broker này vô hình trung lại tạo ra một khoảng thời gian chờ nhất định trước khi luồng dữ liệu được xử lý xong và trả về cho giao diện Frontend. Điều này khiến trải nghiệm truy vấn chưa thể đạt được tốc độ hiển thị tức thời (real-time) trong một số trường hợp câu hỏi quá phức tạp.

Hạn chế thứ hai nằm ở khả năng xử lý các logic nghiệp vụ phức tạp mang tính đặc thù của doanh nghiệp. Ở thời điểm hiện tại, các AI Agent chủ yếu phân tích và sinh mã dựa trên cấu trúc vật lý của cơ sở dữ liệu (Database Schema) được cung cấp. Do chưa được trang bị một lớp ngữ nghĩa (Semantic Layer) hay từ điển thuật ngữ chuyên sâu, AI đôi khi sẽ gặp khó khăn hoặc hiểu sai ý định của người dùng khi họ sử dụng các từ lóng hay thuật ngữ kinh doanh nội bộ. Điển hình như khái niệm "khách hàng tiềm năng", hệ thống AI có thể không tự suy luận được rằng tại công ty này, đó là những khách hàng có trên 3 đơn hàng thành công và tổng chi tiêu vượt quá 5 triệu đồng. Sự thiếu hụt các quy tắc logic nghiệp vụ ngầm định này có thể dẫn đến việc AI sinh ra câu lệnh SQL đúng về mặt cú pháp nhưng lại sai lệch về mặt ý nghĩa kinh doanh.

Cuối cùng, phạm vi hỗ trợ hệ quản trị cơ sở dữ liệu của dự án hiện tại vẫn còn khá hẹp. Toàn bộ kiến trúc cốt lõi, từ bộ lọc bảo mật (Read-only Proxy), cơ chế Connection Pooling cho đến các thiết lập hạ tầng, hiện đang được tối ưu hóa chuyên biệt và hoạt động tốt nhất trên nền tảng PostgreSQL, cụ thể là hệ sinh thái của

Supabase. Hệ thống Agent SQL hiện chưa được mở rộng hỗ trợ để kết nối trực tiếp với các hệ quản trị cơ sở dữ liệu doanh nghiệp phổ biến khác như Microsoft SQL Server, Oracle hay MySQL, từ đó phần nào giới hạn khả năng triển khai linh hoạt của giải pháp tại những công ty đang sử dụng hệ thống cơ sở dữ liệu đa dạng này.

5.3. Hướng phát triển trong tương lai (Future Work)

Định hướng chiến lược dài hạn của dự án là nâng cấp Agent SQL từ một công cụ chuyển đổi ngôn ngữ tự nhiên đơn thuần trở thành một Nền tảng Phân tích Dữ liệu Thông minh (Agentic Data Intelligence Platform). Sự chuyển đổi này tập trung vào việc mang lại các thông tin chuyên sâu thay vì chỉ trả về dữ liệu thô, đồng thời mở rộng quy mô phục vụ cho các bài toán doanh nghiệp phức tạp.

Về mặt **Nâng cao năng lực AI**, hệ thống sẽ được nâng cấp khả năng tương tác với tính năng Hội thoại đa bước (Multi-turn Conversation). Thay vì xử lý từng câu hỏi riêng lẻ, AI Agent sẽ được trang bị bộ nhớ ngữ cảnh (Context memory) để lưu trữ chuỗi hội thoại, cho phép người dùng đặt các câu hỏi nối tiếp nhau một cách tự nhiên. Ví dụ, sau khi trích xuất danh sách khách hàng, người dùng chỉ cần yêu cầu thêm "Trong danh sách đó, hãy lọc ra những người ở Hà Nội", hệ thống sẽ tự động kế thừa truy vấn trước đó để xử lý tiếp. Bên cạnh đó, dự án sẽ xây dựng một Lớp ngữ nghĩa doanh nghiệp (Semantic Layer & Knowledge Graph) sử dụng cơ sở dữ liệu Vector (như pgvector) để làm từ điển thuật ngữ. Lớp ngữ nghĩa này sẽ dạy cho AI hiểu được các khái niệm đặc thù không có sẵn trong cấu trúc bảng, chẳng hạn như định nghĩa chính xác thế nào là một "khách hàng tiềm năng" theo quy chuẩn nội bộ. Đặc biệt, năng lực AI sẽ được đẩy lên tầm cao mới với tính năng Kể chuyện bằng dữ liệu (AI-Powered Data Insights & Narrative). Sau khi nhận kết quả truy vấn, AI sẽ không chỉ hiển thị bảng số liệu khô khan mà còn tự động phân tích để đưa ra các đoạn văn bản nhận xét, tóm tắt các điểm nhấn và xu hướng quan trọng nhất.

Trong khía cạnh **Tối ưu hóa và Mở rộng**, nền tảng sẽ vượt ra khỏi giới hạn của một cơ sở dữ liệu duy nhất thông qua cơ chế Truy vấn đa nguồn (Federated Querying). Bằng việc tích hợp các engine xử lý tốc độ cao như DuckDB tại lớp trung gian, người dùng có thể linh hoạt kết hợp dữ liệu từ Database hệ thống với các tệp tin bên ngoài như Excel, CSV mà họ tự tải lên để phân tích chéo. Hơn nữa, để giải quyết vấn đề hiệu năng truy xuất, AI Agent sẽ được bổ sung khả năng Tự tối ưu SQL (SQL Performance Self-Optimization). Bằng cách tự động đọc và phân tích các báo cáo thực thi (như EXPLAIN ANALYZE), AI có thể nhận diện các câu lệnh chạy chậm và chủ động đề xuất giải pháp viết lại mã lệnh hiệu quả hơn hoặc khuyên quản trị viên bổ sung thêm các Index cần thiết.

Đối với các **Tính năng hướng người dùng**, giao diện hệ thống sẽ được nâng cấp thành một Dashboard tương tác toàn diện. Người dùng có thể tự do lưu các truy

vấn phổ biến thành các biểu đồ, kéo thả các widget (công cụ hiển thị) để tạo thành một bảng điều khiển cá nhân hóa, đồng thời sử dụng các bộ lọc động để thu hẹp phạm vi dữ liệu mà không cần phải viết lại câu hỏi. Trải nghiệm người dùng còn được nâng tầm với hệ thống Cảnh báo chủ động (Proactive Alerting), cho phép thiết lập các mốc theo dõi hoàn toàn bằng ngôn ngữ tự nhiên. Người dùng có thể ra lệnh như "Báo cho tôi qua Slack nếu doanh thu hôm nay giảm quá 20%", hệ thống sẽ tự động chạy các tác vụ ngầm định kỳ và kích hoạt cảnh báo thông qua Webhook khi điều kiện được thỏa mãn.

Cuối cùng, về **Hạ tầng và Bảo mật**, nền tảng sẽ triển khai công nghệ OpenTelemetry để theo dõi vết (tracing) toàn diện. Việc này giúp quản trị viên giám sát chặt chẽ luồng dữ liệu đi từ API Gateway qua các dịch vụ NL2SQL và Query Service, từ đó dễ dàng dò tìm nút thắt cổ chai và tối ưu hóa hiệu năng. *Bên cạnh đó, để đáp ứng tiêu chuẩn bảo mật tối đa (như bạn đã định hướng trong yêu cầu), dự án sẽ lên kế hoạch thử nghiệm triển khai các mô hình LLM mã nguồn mở (chẳng hạn như Llama 3) chạy hoàn toàn trên hạ tầng máy chủ riêng (on-premise). Việc chuyển dịch từ các API bên thứ ba (như hiện tại đang dùng OpenRouter/Gemini) sang mô hình nội bộ sẽ đảm bảo dữ liệu doanh nghiệp không bao giờ phải truyền tải ra bên ngoài hệ thống internet.*

TÀI LIỆU THAM KHẢO

- [1] [FastAPI Documentation](#), “FastAPI framework, high performance, easy to learn, fast to code, ready for production,” Accessed: May 16, 2026.
- [2] [Next.js 14 Documentation](#), “The React Framework for the Web,” Vercel, Accessed: May 16, 2026.
- [3] [Apache Kafka Documentation](#), “A distributed streaming platform,” The Apache Software Foundation, Accessed: May 16, 2026.
- [4] [Supabase Documentation](#), “The Open Source Firebase Alternative (PostgreSQL, Auth, Edge Functions),” Accessed: May 16, 2026.
- [5] [Google Gemini API Documentation](#), “Build with Gemini models on Google AI Studio,” Google for Developers, Accessed: May 16, 2026.
- [6] [CrewAI Framework Documentation](#), “Cutting-edge framework for orchestrating role-playing, autonomous AI agents,” Accessed: May 16, 2026.
- [7] [LangChain Documentation](#), “Building applications with LLMs through composability,” Accessed: May 16, 2026.
- [8] Spider: A Large-Scale Hierarchical Semantic Parsing and Text-to-SQL Dataset, T. Yu *et al.*, “Spider: A Large-Scale Hierarchical Semantic Parsing and Text-to-SQL Dataset on Cross-Domain Databases,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [9] [OWASP SQL Injection Prevention Cheat Sheet](#), “SQL Injection Prevention Cheat Sheet,” OWASP Foundation, Accessed: May 16, 2026.
- [10] [Trivy Documentation](#), “A Comprehensive Security Scanner for Containers and other Artifacts,” Aqua Security, Accessed: May 16, 2026.
- [11] [k6 Performance Testing Documentation](#), “Open-source load testing tool and SaaS for engineering teams,” Grafana Labs, Accessed: May 16, 2026.
- [12] [Docker Documentation](#), “Run your app as a non-root user,” Accessed: May 16, 2026.